MySQL & PostgreSQL Professional DBA Roadmap

Oct 31, 2026 · @SHERIF MOHAMED

Part 0 — How to use this roadmap

You already have the hands-on muscle memory. The gap is the mental model: why each step exists and what breaks without it. So every module here follows one pattern: **Concept (why) → Lab (how) → Break it (what failure looks like) → Verify (how you prove it works)**. The "break it" step is what turns copied commands into real understanding.

Versions to target (as of Oct 2026)

Engine	Lab baseline	Also learn	Why
MySQL	8.4 LTS (8.4.13)	9.7 LTS (released Apr 2026)	8.4 has the most mature tooling (XtraBackup 8.4 is GA; XtraBackup 9.7 was still a release candidate in mid-2026). 9.7 is the next production target. MySQL 8.0 reached end of life in Apr 2026.
PostgreSQL	18 (18.6)	19 (GA planned 19 Nov 2026)	18 is the current stable major. PostgreSQL 14 stops receiving fixes on 12 Nov 2026, so practise a 14/16 → 18 upgrade.

LTS lines cannot be skipped in MySQL upgrades: 8.0 → 8.4 → 9.7. PostgreSQL major upgrades can jump versions with `pg_upgrade`.

Lab setup (build once, reuse in every part)

1. **Three Rocky Linux 9 VMs + one Ubuntu 24.04 control node** (Vagrant + VirtualBox/libvirt, or Proxmox). Three nodes is the minimum for quorum (Group Replication, etcd, Patroni).
2. **Separate disks per VM** for data and for WAL/binlogs. This lets you practise disk layout and see I/O effects.
3. **Snapshots** before every risky lab, so you can break things freely.
4. **Docker** only for quick throwaway tests (a version, a query). Do the admin labs on VMs, because systemd, SELinux, firewalls and filesystems are part of the job.
5. **A sample dataset**: MySQL `employees` and `sakila` databases; PostgreSQL `pagila`, plus `pgbench` and `sysbench` for load.
6. **A notebook (git repo)**: for every lab, write down the command, why you ran it, and what you saw. That repo later becomes your Ansible code and your portfolio.

Levels — know where you are

Level	You can…
1 Beginner	Install, connect, create users and databases, read the config and the logs

Level	You can…
2 Junior DBA	Explain the architecture, tune basic memory, take and **restore** backups, set up replication
3 DBA	Do PITR, run HA failover drills, tune queries with EXPLAIN, design indexes, automate with Ansible
4 Senior DBA	Plan capacity, do zero-downtime upgrades, run a disaster-recovery strategy, find root causes of incidents, set standards

Pace: 52 weeks at 8–10 h/week

Quarter	Weeks	Focus	Milestone at the end
Q1	1–13	Lab, foundations, Part 1 administration (single servers), Ansible basics	Both engines rebuilt from scratch by Ansible, hardened, monitored
Q2	14–26	Replication & HA, failure drills, upgrades, start of Part 2	Working InnoDB Cluster and Patroni cluster, every failure drill written up
Q3	27–39	Part 2 programming, Part 3 backup & restore, Ansible roles	Automated backups + a weekly automated restore test
Q4	40–52	Ansible day-2, Part 5 performance, partitioning & sharding, SQL Server appendix, capstone	Capstone repo published, CV and interview prep done

The week-by-week plan follows Part 0. Each week is about 2–3 h of video, 5–6 h of labs and 1 h of notes.

For each item, the **official manual** is your primary source: MySQL 8.4 Reference Manual and PostgreSQL documentation. Blogs and videos are fine for a second explanation, but always check them against the manual.

52-week plan

One row per week. "Output" is what must exist in your git notebook before you move on. Weeks 13, 26, 39 and 52 are review-and-milestone weeks; if you fall behind, use them to catch up rather than skipping topics.

Two rules for every week: write down *why* before running a command, and break something on purpose before you call the topic done.

Q1 — Foundations and single-server administration (weeks 1–13)

Week	Focus (roadmap section)	Output	Watch
1	Lab build, study method (Part 0)	3 Rocky VMs + Ubuntu control node, snapshots, git notebook repo	Hussein Nasser: ACID and transactions

Week	Focus (roadmap section)	Output	Watch
2	Architecture of both engines (1.1)	Both architectures drawn from memory and matched to `ps` / thread lists	CMU 15-445: storage, buffer pools
3	MySQL install + OS prep on Rocky and Ubuntu (1.2)	OS checklist applied; MySQL 8.4 from the vendor repo; file-layout notes	Imran Afzal MySQL course: install sections
4	PostgreSQL install on both OSes; move data dirs with SELinux on (1.2)	PG 18 from PGDG on both OSes; datadir on `/data`; the "wrong owner" failure captured	Sadiq Basha Part 1: architecture, install
5	Configuration (1.3)	One dynamic and one restart-only change per engine, each with proof of the active value	Imran Afzal: configuration
6	MySQL security: accounts, roles, TLS, password policy (1.4)	Role pattern (owner, app_rw, app_ro, monitor, backup) on `sakila`	Becoming a Production MySQL DBA: user administration
7	PostgreSQL security: roles, ownership, default privileges, `pg_hba.conf` (1.4)	Same role pattern on `pagila`; the first-match `pg_hba` break documented	Sadiq Basha Part 1: security
8	Transactions, MVCC, isolation, locking, collations (1.5)	Lost update, deadlock and lock wait reproduced in both engines and captured	Hussein Nasser: isolation, MVCC; CMU: concurrency control
9	Monitoring and logging (1.6)	PMM (or Prometheus + Grafana) on both; slow logs + `pg_stat_statements` on	Percona webinars on PMM
10	Routine maintenance: vacuum, analyze, online DDL, metadata locks (1.7)	Bloat lab with `pgbench`; metadata-lock lab in MySQL	Scaling Postgres: vacuum episodes
11	Ansible basics: inventory, ad-hoc, playbooks (4.1–4.2)	Playbook that applies the week 3 OS checklist	Jeff Geerling Ansible 101, episodes 1–4
12	Ansible: variables, handlers, templates, Vault, roles (4.2, 4.4)	`os_prep`, `mysql_install`, `pg_install` roles	Ansible 101, episodes 5–6

Week	Focus (roadmap section)	Output	Watch
13	**Review + milestone 1**	Both servers destroyed and rebuilt by Ansible; Part 1 self-check (1.1–1.7) answered in writing	Rewatch any weak sections

Q2 — Replication, HA and upgrades (weeks 14–26)

Week	Focus (roadmap section)	Output	Watch
14	MySQL async replication, ROW format, GTID (1.8)	Source + replica with GTID auto-positioning, 8.4 syntax	MySQL Replication zero-to-hero
15	Semi-sync, replica hygiene, parallel applier, manual failover (1.8)	Manual failover runbook, tested twice	Becoming a Production MySQL DBA: replication
16	Group Replication internals: consensus, certification, flow control (1.8)	3-node single-primary group; quorum loss reproduced	Percona "InnoDB Cluster in a Nutshell" Part 1
17	InnoDB Cluster: MySQL Shell AdminAPI, Router, clone (1.8)	Your old cluster rebuilt with every step explained; a node re-provisioned with clone	InnoDB Cluster in a Nutshell Part 2
18	PostgreSQL streaming replication, slots, synchronous mode (1.8)	Primary + 2 standbys; slot disk-fill danger shown, then fixed with `max_slot_wal_keep_size`	Sadiq Basha Part 2
19	`pg_rewind`, cascading and logical replication (1.8)	Old primary rejoined with `pg_rewind`; one table replicated logically	Sadiq Basha Part 2: logical replication
20	etcd + Patroni, rebuilt with understanding (1.8)	3-node Patroni; leader-key/TTL behaviour written up	Patroni course (Sandro Servino)
21	HAProxy, PgBouncer, MySQL Router routing (1.8)	Apps connect through a proxy in both stacks; failover visible to a client	Patroni course: routing
22	Failure drills, both stacks (1.8)	All 6 drills: prediction vs actual result, in writing	—
23	MySQL upgrades: 8.0 → 8.4 rolling (1.9)	Upgrade checker report + rolling upgrade on a replicated pair	Imran Afzal: upgrades

Week	Focus (roadmap section)	Output	Watch
24	PostgreSQL upgrades: 16 → 18 with `pg_upgrade --link`, in Patroni (1.9)	Upgrade runbook with rollback plan	Scaling Postgres: upgrade episodes
25	Runbooks, capacity planning, postmortems (1.10); SQL fluency (2.1)	5 runbooks; CTE / window-function / upsert exercises	Stephen Grider: SQL foundations
26	**Review + milestone 2**	Both clusters stable; drill report; Part 1 self-check fully answered	—

Q3 — Programming objects, backup & restore, Ansible roles (weeks 27–39)

Week	Focus (roadmap section)	Output	Watch
27	Views, materialized views, functions, security definer (2.2–2.4)	Reporting view + summary in both engines; least-privilege access proved	Stephen Grider: views, functions
28	Procedures, triggers, scheduling, managing DB code (2.5–2.8)	HR mini-system (2.8 lab project) in MySQL	Grider: transactions; Ami Levin: window functions
29	Same project in PostgreSQL; deploy with a migration tool (2.8)	HR system in PG, deployed via Flyway or Sqitch	—
30	Backup concepts; logical tools: mysqldump, MySQL Shell, mydumper, pg_dump (3.1–3.3)	Logical backup + single-table restore in both engines	Hussein Nasser: backups
31	XtraBackup: full, incremental, prepare, restore (3.2)	Full + 2 incrementals restored to a new VM, timed	MySQL Replication zero-to-hero: XtraBackup
32	Binlog streaming + MySQL PITR (3.2)	DROP TABLE recovered to one second before the drop	—
33	pg_basebackup + WAL archiving + manual PITR (3.3)	Manual PITR without any extra tool, to understand `recovery.signal`	Sadiq Basha: backup and PITR

Week	Focus (roadmap section)	Output	Watch
34	pgBackRest with Patroni + MinIO (S3) (3.3)	Stanza on the cluster; PITR with `--delta`; backup from a standby	Postgres FM: pgBackRest episode
35	Backup jobs: scripts, systemd timers, alerts (3.4–3.5)	Jobs on both stacks; "no backup in 26 h" alert fires in a test	—
36	Restore labs 1–7 (3.7)	Restore report with real RTO numbers; automated weekly restore test	—
37	Ansible layout, collections, Molecule (4.3, 4.8)	Repo structure; first role passing `molecule test`	Ansible 101, episodes 7–8
38	MySQL roles: users, replication, InnoDB Cluster, backup (4.5, 3.6)	One playbook builds the MySQL stack with backups	—
39	**Milestone 3:** PostgreSQL roles: pg_hba, users, etcd, Patroni, pgBackRest (4.6, 3.6)	One playbook builds the PG stack; second run reports `changed=0`	—

Q4 — Day-2 automation, performance, scale-out, SQL Server, capstone (weeks 40–52)

Week	Focus (roadmap section)	Output	Watch
40	Day-2 playbooks, CI, AWX/Semaphore (4.7, 4.9)	Rolling restart, minor upgrade and user-provisioning playbooks	Ansible 101, episodes 10–12
41	Index internals and types (5.1–5.2)	10M-row lab: no index vs single vs composite vs covering	Hussein Nasser: indexing, B-trees
42	Index design + reading plans (5.3–5.4)	20 slow queries, each with a one-line plan diagnosis	Dan Sullivan: query tuning
43	Statistics + query rewriting (5.5–5.6)	Correlated-column wrong plan fixed; keyset pagination demo	Lucian Oprea: PG tuning
44	Server tuning + benchmarking (5.7, 5.9)	Bad-config vs tuned-config numbers with sysbench and pgbench; PgBouncer test	Postgres Performance Demystified
45	Partitioning in both engines (5.8.1–5.8.3)	50M-row table partitioned; DELETE vs DROP/DETACH measured; pg_partman automation	Hussein Nasser: partitioning

Week	Focus (roadmap section)	Output	Watch
46	Sharding concepts + Citus (5.8.4, 5.8.6)	Citus 1 + 2 workers; rebalance after adding a worker	Hussein Nasser: sharding
47	Vitess (5.8.5)	Vitess local example resharded under load, with no lost writes	Vitess docs videos
48	SQL Server: install, config, security, T-SQL objects (Appendix A.1–A.5)	SQL Server 2025 Developer in Docker + a Windows VM; role pattern recreated	Microsoft Learn: SQL Server admin path
49	SQL Server: backup/restore, Availability Groups, performance (Appendix A.6–A.9)	PITR with log backups; 2-node AG; First Responder Kit health check	Brent Ozar free videos
50	Capstone build (Part 6)	Everything from Part 6 built by one Ansible run	—
51	Capstone drills: DR, major upgrades 8.4 → 9.7 and 18 → 19, performance report	Drill and upgrade reports; top-10 query before/after report	—
52	**Final milestone:** portfolio, CV, interview prep, certification plan	Public repo README, CV bullets with numbers, 50 interview Q&As in your own words	Practice exams for your chosen certification

Part 1 — Administration fundamentals (beginner → advanced)

Part 1 rebuilds everything you have already done, but starting from the internals. Each module gives the *why* first, then the MySQL and PostgreSQL specifics, then a lab and a failure to cause on purpose.

1.1 Architecture — the model everything else rests on (Level 1–2)

Why: almost every admin decision (memory sizing, pooling, vacuum, backups, replication) follows from how the engine stores and writes data. Learn this first and the later commands stop being magic.

- **MySQL:** one `mysqld` process with one thread per connection. The SQL layer (parser, optimizer, executor) sits on top of pluggable storage engines; in practice that means **InnoDB**.
    - InnoDB keeps pages in the **buffer pool**. Changes are written first to the **redo log** (crash recovery), old row versions go to **undo logs** (MVCC + rollback), and the **doublewrite buffer** protects against torn pages.
    - The **binary log** is separate. It belongs to the server, not to InnoDB, and is used for replication and point-in-time recovery. Redo = "repair this server after a crash"; binlog = "replay these changes somewhere else".

- **PostgreSQL:** the `postgres` postmaster forks **one OS process per connection**. Background processes include the checkpointer, background writer, WAL writer, autovacuum, WAL senders/receivers, the archiver and (in 18) I/O workers.
    - Pages live in **shared_buffers** plus the OS page cache. Every change is written to the **WAL** first. WAL is used for crash recovery *and* for replication and PITR, so PostgreSQL has a single log where MySQL has two.
    - MVCC keeps old row versions **inside the table itself**. That is why `VACUUM` exists and why InnoDB has "purge" instead.
    - One **cluster** (= one instance, one data directory) holds many databases; each database holds schemas.
- **Why this matters:** a process per connection makes PostgreSQL connections expensive, so you need PgBouncer. In-table versions explain bloat. Two logs in MySQL explain `sync_binlog` + `innodb_flush_log_at_trx_commit`.
- **Lab:** draw both architectures from memory. Then run `ps -ef | grep postgres` and `SHOW PROCESSLIST` / `performance_schema.threads` and match each process or thread to your drawing.

1.2 Installation, OS preparation and file layout (Level 1)

Why: distro packages often ship old versions. The vendor repositories (MySQL APT/YUM repo, PGDG repo) let you pick and pin the exact major version. The OS settings matter as much as the DB settings.

- **OS checklist (both engines):** XFS or ext4 with `noatime`; data and logs on separate volumes; `vm.swappiness=1`; **Transparent Huge Pages off**; raised `nofile`/`nproc` limits; time sync (chrony); firewall ports 3306/33060/33061 or 5432; SELinux left **enforcing**, with correct labels (`semanage fcontext` + `restorecon`) when you move a data directory.
- **MySQL:** `mysqld --initialize` creates the data directory and a temporary root password (in the error log on RHEL). Know where things live: datadir `/var/lib/mysql`, config `/etc/my.cnf` (Rocky) or `/etc/mysql/` (Ubuntu), error log, socket, PID file.
- **PostgreSQL:** on Rocky, `postgresql-18-setup initdb` creates `/var/lib/pgsql/18/data`. On Ubuntu, `pg_createcluster` puts config in `/etc/postgresql/18/main` and data in `/var/lib/postgresql/18/main`; learn `pg_lsclusters` and `pg_ctlcluster`. Since PostgreSQL 18, `initdb` enables **data checksums** by default. Checksums detect silent disk corruption.
- **Lab:** install both from vendor repos on Rocky *and* Ubuntu, then move the data directory to `/data` with SELinux enforcing. Write down every file you had to touch.
- **Break it:** start the server with the wrong owner on the data directory, or with no SELinux label. Read the exact error in the log and in `journalctl -u`.

1.3 Configuration (Level 1–2)

Why: you must know where a setting comes from, whether changing it needs a restart, and how to prove which value is active.

- **MySQL:** option files are read in a fixed order (`mysqld --verbose --help | head` shows it). `SET PERSIST` writes to `mysqld-auto.cnf`, which overrides `my.cnf`. Use `performance_schema.variables_info` to see where each value came from. Variables are either *dynamic* or need a restart.

- **PostgreSQL:** `postgresql.conf` → `include_dir` → `postgresql.auto.conf` (written by `ALTER SYSTEM`; the last one wins). `pg_settings.context` tells you what a change needs: `postmaster` = restart, `sighup` = reload (`SELECT pg_reload_conf()`), `user` = per session. `pg_settings.pending_restart` shows changes waiting for a restart.
- **The first parameters to understand:**

Purpose	MySQL	PostgreSQL
Main cache	`innodb_buffer_pool_size` (50–75% RAM on a dedicated host)	`shared_buffers` (~25% RAM) + `effective_cache_size` (~50–75%)
Crash-log size	`innodb_redo_log_capacity`	`max_wal_size`, `checkpoint_timeout`
Per-operation memory	`sort_buffer_size`, `join_buffer_size` (keep small)	`work_mem` (per sort/hash node!), `maintenance_work_mem`
Connections	`max_connections`	`max_connections` (keep low, pool instead)
Durability	`innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`	`fsync=on`, `synchronous_commit=on`, `full_page_writes=on`
Replication	`server_id`, `log_bin`, `gtid_mode=ON`, `enforce_gtid_consistency=ON`	`wal_level=replica` or `logical`, `max_wal_senders`

- **Lab:** change one dynamic and one restart-only parameter in each engine, and prove the active value each time.

1.4 Security: users, roles, privileges, authentication (Level 1–3)

Why: "just GRANT ALL" is the most common cause of incidents and audit failures. The goal is **least privilege** and accounts that are easy to audit.

- **MySQL:**
  - An account is `'user'@'host'`. `app@'10.0.%'` and `app@'localhost'` are *different accounts*, with different passwords and privileges. This explains most "access denied" puzzles.
  - Default authentication is `caching_sha2_password`. `mysql_native_password` is disabled by default in 8.4 and removed in 9.x, so old clients and old blog posts will fail.
  - Privileges exist at global, database, table, column and routine level. Use **roles** (`CREATE ROLE`, `GRANT role TO user`, `SET DEFAULT ROLE`) instead of granting to each user.
  - Hardening: the `validate_password` component, `PASSWORD EXPIRE`, `FAILED_LOGIN_ATTEMPTS` + `PASSWORD_LOCK_TIME`, `require_secure_transport=ON` (TLS), removing anonymous and remote-root accounts. `SHOW GRANTS FOR` is your audit tool.
- **PostgreSQL:**

    - Users and groups are both **roles**; a "user" is just a role with `LOGIN`. Every object has an **owner**, and the owner can do anything with it. Ownership is the most misunderstood part of PostgreSQL security.
    - Privileges stack in layers: `CONNECT` on the database → `USAGE` on the schema → `SELECT/INSERT/…` on tables → `USAGE` on sequences. Missing any one layer means access is denied.
    - `GRANT … ON ALL TABLES` covers only existing tables. **`ALTER DEFAULT PRIVILEGES`** covers tables created later. Forgetting this is the classic "new table, app broke" incident.
    - Since PG 15, ordinary users can no longer create objects in the `public` schema by default. Predefined roles: `pg_read_all_data`, `pg_write_all_data`, `pg_monitor`, `pg_maintain` (17+).
    - **`pg_hba.conf`** decides *who may connect, from where, and how*. Lines are checked top to bottom and the **first match wins**. Use `scram-sha-256`; `md5` is deprecated as of PG 18.
    - Advanced: Row-Level Security policies, `pgaudit` for audit logging, SSL with client certificates.
- **A standard role pattern (both engines):** an owner/migration account that owns the schema, `app_rw` for the application, `app_ro` for reporting, a `monitor` account, a `backup` account with only the privileges its tool needs, and named DBA accounts (no shared root/postgres logins).
- **Lab:** build that pattern in both engines for the `sakila`/`pagila` databases. Prove with a new table that `app_ro` can read it without a new GRANT.
- **Break it:** put a broad `host all all 0.0.0.0/0 trust` line *above* your strict rule in `pg_hba.conf`, and see that first-match makes your strict rule useless.

1.5 Storage, transactions, MVCC and locking (Level 2–3)

Why: most production incidents (lock pile-ups, deadlocks, bloat, a server that "hangs") are transaction problems, not hardware problems.

- **Isolation:** MySQL/InnoDB defaults to **REPEATABLE READ**; PostgreSQL defaults to **READ COMMITTED**. The same application code can behave differently on each.
- **Locking:** InnoDB uses row locks plus **gap and next-key locks** under REPEATABLE READ, which prevents phantoms but surprises people who expect only row locks. PostgreSQL row locks are stored in the row itself, so readers never block writers.
- **Long transactions are the enemy:** in InnoDB they grow the undo *history list length* and slow everything. In PostgreSQL they hold back the vacuum horizon and cause bloat. Learn to find and kill them.
- **Deadlocks:** `SHOW ENGINE INNODB STATUS` (LATEST DETECTED DEADLOCK) and `innodb_print_all_deadlocks`. In PostgreSQL: `log_lock_waits`, `deadlock_timeout`, `pg_locks` joined to `pg_stat_activity`, and `pg_blocking_pids()`.
- **Physical storage:** InnoDB file-per-table `.ibd` files, the system and undo tablespaces. PostgreSQL stores each table as files under `base/<db oid>/`, with TOAST for large values; a PostgreSQL tablespace is just a directory.
- **Character sets and collations:** use `utf8mb4` in MySQL. In PostgreSQL, choose the encoding and collation at `initdb`. **A glibc upgrade can change sort order and silently corrupt indexes on text columns.** This is why ICU collations exist and why you `REINDEX` after some OS upgrades.

- **Lab:** open two sessions and reproduce a lost update, a deadlock and a lock wait in both engines. Then capture each from the monitoring views.

1.6 Monitoring, logging and observability (Level 2–3)

Why: a DBA's value is noticing problems *before* users do, and proving the root cause afterwards.

- **MySQL:** error log, **slow query log** (`long_query_time`, `log_queries_not_using_indexes` with care), `performance_schema`, the **sys schema** (`sys.statement_analysis`, `sys.schema_unused_indexes`, `sys.innodb_lock_waits`), `information_schema`, `SHOW ENGINE INNODB STATUS`. Never leave the general log on in production.
- **PostgreSQL:** set up logging properly: `log_line_prefix`, `log_min_duration_statement`, `log_checkpoints`, `log_autovacuum_min_duration`, `log_lock_waits`, `log_temp_files`. Views: `pg_stat_activity`, `pg_stat_database`, `pg_stat_user_tables`, `pg_stat_io` (16+), `pg_locks`, `pg_stat_replication`. Install the **`pg_stat_statements`** extension on every server.
- **Tooling:** Percona Monitoring and Management (PMM) covers both engines. Alternatively Prometheus + `mysqld_exporter`/`postgres_exporter` + Grafana. For log analysis: `pt-query-digest` (MySQL) and `pgBadger` (PostgreSQL).
- **What to alert on:** instance down, replication lag or broken replication, connections above 80% of max, disk above 80%, long-running and idle-in-transaction sessions, deadlock rate, checkpoint frequency, and transaction ID age (PostgreSQL).

1.7 Routine maintenance (Level 2–3)

Why: databases degrade quietly. Statistics go stale, tables bloat, logs fill disks.

- **MySQL:** `ANALYZE TABLE` and histograms (`ANALYZE TABLE … UPDATE HISTOGRAM`), `OPTIMIZE TABLE` (a full rebuild, use rarely), binlog retention (`binlog_expire_logs_seconds`), **online DDL** (`ALGORITHM=INSTANT|INPLACE, LOCK=NONE`), and **metadata locks** (why an `ALTER` can freeze a busy table). Large schema changes: `gh-ost` or `pt-online-schema-change`.
- **PostgreSQL:** `VACUUM` reclaims dead row versions; `ANALYZE` refreshes planner statistics; **autovacuum** does both and must be tuned per table on big tables (`autovacuum_vacuum_scale_factor`, cost limits).
    - **Transaction ID wraparound:** XIDs are 32-bit. Rows must be "frozen" before about 2 billion transactions pass, or the database stops accepting writes to protect itself. Monitor `age(datfrozenxid)`.
    - Use `CREATE INDEX CONCURRENTLY` and `REINDEX CONCURRENTLY`; remove bloat with `pg_repack`. PostgreSQL 19 adds a built-in `REPACK (CONCURRENTLY)` command.
    - Safety settings for every DDL session: `lock_timeout`, `statement_timeout`, `idle_in_transaction_session_timeout`.
- **Lab:** generate bloat with `pgbench`, watch `n_dead_tup` grow, then vacuum it away. In MySQL, run an `ALTER` while a long transaction is open and watch it wait on a metadata lock.

1.8 Replication and high availability — revisit what you built, with the why (Level 2–4)

Why: you have built these setups. Now you need to explain each setting and predict what happens during a failure.

MySQL

- **Asynchronous replication:** the source writes the binlog; the replica's I/O thread copies it into a relay log; the SQL/applier threads replay it.
    - Use `binlog_format=ROW`, because statement-based replication can give different results on the replica.
    - Use **GTIDs**: each transaction gets a global ID, so a replica can find its position automatically after a failover (`SOURCE_AUTO_POSITION=1`).
    - Commands in 8.4+: `CHANGE REPLICATION SOURCE TO`, `START REPLICA`, `SHOW REPLICA STATUS`. The old MASTER/SLAVE syntax is removed.
- **Semi-synchronous replication:** the source waits until at least one replica *received* the transaction. This reduces data loss on failover; it does not mean the replica has applied it.
- **Replica hygiene:** `super_read_only=ON`, `relay_log_recovery=ON`, multi-threaded applier (`replica_parallel_workers` + `replica_preserve_commit_order`).
- **Group Replication:** nodes agree on every transaction through a Paxos-based consensus, then each node *certifies* it for conflicts.
    - A majority is required, which is why you need 3 nodes: 2 nodes cannot lose one and keep a majority.
    - Requirements: InnoDB tables only, a primary key on every table, GTIDs. Learn flow control, and prefer single-primary mode.
- **InnoDB Cluster** = Group Replication + MySQL Shell AdminAPI (`dba.createCluster()`, `cluster.status()`) + **MySQL Router** (port 6446 read/write, 6447 read-only). **ClusterSet** adds a disaster-recovery cluster in another site. **ReplicaSet** is async replication managed by the same AdminAPI. The **clone plugin** provisions new members.
- **Alternatives to know about:** ProxySQL (query routing), Orchestrator (async topology failover), Percona XtraDB Cluster (Galera).

PostgreSQL

- **Streaming replication:** standbys replay the primary's WAL byte for byte, so the whole cluster is copied, not individual databases.
    - **Replication slots** stop the primary from deleting WAL a standby still needs. The danger: an abandoned slot fills the disk. Set `max_slot_wal_keep_size`.
    - `synchronous_standby_names` sets zero-data-loss mode. The trade-off: writes stop if no synchronous standby answers.
    - **`pg_rewind`** lets a failed old primary rejoin as a standby without a full re-copy. It needs `wal_log_hints=on` or checksums.
- **Logical replication:** publications and subscriptions replicate individual tables. It is the tool for low-downtime major upgrades and partial copies. It does **not** replicate DDL, so schema changes must be applied on both sides.
- **Patroni (your build):**
    - Each node runs a Patroni agent. The leader holds a **leader key in etcd with a TTL** and must keep renewing it. If it cannot reach etcd, it **demotes itself**, which prevents two primaries (split brain).
    - etcd needs 3 nodes for its own Raft quorum.

    - Learn `patronictl list | switchover | failover | edit-config`, the difference between DCS settings and local settings, the watchdog, and routing with HAProxy health checks on Patroni's REST API (`/primary`, `/replica`) or with vip-manager. Put PgBouncer in front for pooling.

Failure drills (do every one in both stacks): kill the primary process; power off the primary VM; cut the network between nodes (iptables); fill the disk; stop etcd on two nodes; do a planned switchover. For each, write down the expected behaviour first, then compare it with what really happened.

1.9 Upgrades and patching (Level 3–4)

Why: upgrades are where data loss and long outages happen. A professional DBA has a tested, repeatable procedure with a rollback plan.

- **MySQL:** run `util.checkForServerUpgrade()` in MySQL Shell first. Then do an in-place upgrade (the server upgrades the data dictionary on start), or a rolling upgrade: replicas first, then a switchover. Don't skip LTS lines (8.0 → 8.4 → 9.7).
- **PostgreSQL:** minor versions = new binaries + restart. Major versions:
    - `pg_upgrade --check`, then `pg_upgrade --link` (minutes instead of hours).
    - Or logical replication into a new cluster, for near-zero downtime.
    - Afterwards: update extensions and refresh statistics. Since PG 18, `pg_upgrade` keeps optimizer statistics.
- **Lab:** upgrade MySQL 8.0 → 8.4 in a replication pair, and PostgreSQL 16 → 18 with `pg_upgrade --link`, inside your Patroni cluster.

1.10 Professional practice (Level 3–4)

- Write a **runbook** for every recurring task (failover, restore, adding a replica, user requests) and test it.
- Capacity planning: track data growth, connections and IOPS monthly and forecast disk needs 6–12 months ahead.
- Change management: every change has a reason, a rollback plan and a verification step.
- Write a blameless postmortem after every incident.

Part 1 self-check — can you explain, without notes…

- Why does PostgreSQL need `VACUUM` while MySQL does not have a direct equivalent?
- Why do binlogs and redo logs both exist in MySQL?
- Why does a 2-node Group Replication or etcd cluster fail when one node dies?
- What exactly stops two Patroni nodes from both becoming primary?
- Why can a new table break an application in PostgreSQL even though you ran `GRANT … ON ALL TABLES`?

Part 2 — Database programming objects

A DBA doesn't need to write the application, but you do need to read, review, secure, deploy and troubleshoot code that lives inside the database. Learn each object in this order, in both engines.

2.1 SQL you must be fluent in first

- Joins, `GROUP BY`/`HAVING`, subqueries, `EXISTS` vs `IN`.
- **CTEs** (`WITH`) and **recursive CTEs** (org charts, hierarchies).
- **Window functions:** `ROW_NUMBER()`, `RANK()`, `LAG()/LEAD()`, running totals with `SUM() OVER (…)`.
- **Upserts:** MySQL `INSERT … ON DUPLICATE KEY UPDATE`; PostgreSQL `INSERT … ON CONFLICT … DO UPDATE` and `MERGE` (15+); PostgreSQL `RETURNING`.
- **Constraints and generated columns:** `CHECK`, foreign keys, `GENERATED ALWAYS AS (…)`. PostgreSQL also has identity columns, domains and custom types.

2.2 Views

Why: a view hides complexity and limits what users can see (columns and rows) without copying data.

- Both engines support simple **updatable views** and `WITH CHECK OPTION`, which stops users inserting rows the view would not show.
- **Security model:**
    - MySQL views have `SQL SECURITY DEFINER` (the default; runs with the creator's privileges) or `INVOKER`. If the `DEFINER` account is dropped, the view breaks. That is a common error after migrations.
    - PostgreSQL views run with the **view owner's** privileges by default. `WITH (security_invoker = true)` (15+) runs them with the caller's privileges. `security_barrier` prevents data leaking through a crafted function in a `WHERE` clause.
- **Lab:** give `app_ro` access to a view of `employees` without salaries, with no access to the base table.

2.3 Materialized views and summary tables

Why: they precompute expensive aggregates for reports.

```
-- PostgreSQL: native materialized view
CREATE MATERIALIZED VIEW mv_sales_by_month AS
  SELECT date_trunc('month', payment_date) AS month, sum(amount) AS total
  FROM payment GROUP BY 1;
CREATE UNIQUE INDEX ON mv_sales_by_month (month);        -- required for CONCURRENTLY
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_sales_by_month; -- readers are not blocked

-- MySQL: no materialized views -> summary table refreshed by an event
CREATE TABLE sales_by_month (month DATE PRIMARY KEY, total DECIMAL(14,2));
CREATE EVENT ev_refresh_sales ON SCHEDULE EVERY 1 HOUR DO
  REPLACE INTO sales_by_month
  SELECT DATE_FORMAT(payment_date, '%Y-%m-01'), SUM(amount) FROM payment GROUP BY 1;
```

2.4 Functions

Why: reusable calculations that you call inside SQL. Function attributes change how the database treats your code, so they matter for correctness and performance.

```
-- MySQL
CREATE FUNCTION full_name(f VARCHAR(50), l VARCHAR(50)) RETURNS VARCHAR(101)
```

```
  DETERMINISTIC NO SQL
  RETURN CONCAT(f, ' ', l);

-- PostgreSQL (SQL-standard body, 14+)
CREATE FUNCTION full_name(f text, l text) RETURNS text
  LANGUAGE sql IMMUTABLE PARALLEL SAFE
  RETURN f || ' ' || l;
```

- **MySQL:** with binary logging on, a function must be declared `DETERMINISTIC`, `NO SQL` or `READS SQL DATA`, or you must set `log_bin_trust_function_creators`. **Why:** a non-deterministic function could give a different result on a replica.
- **PostgreSQL volatility** (`IMMUTABLE` / `STABLE` / `VOLATILE`) tells the planner whether it can cache the result or use the function in an index. Mark it wrong and you get wrong results or slow plans.
  - Functions can `RETURNS TABLE (…)` / `SETOF`, so they can act like parameterised views.
  - PostgreSQL also supports PL/pgSQL, PL/Python and other languages.
- **`SECURITY DEFINER`** (both engines) runs the function with the owner's privileges. In PostgreSQL, *always* add `SET search_path = pg_catalog, public` (or your schema). **Why:** otherwise a user can create an object with the same name in their own schema and hijack the function's privileges.

2.5 Stored procedures

Why: multi-step operations that run on the server, with transaction control and error handling.

```
-- MySQL
DELIMITER //
CREATE PROCEDURE transfer(IN p_from INT, IN p_to INT, IN p_amount DECIMAL(12,2))
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;            -- pass the original error to the caller
  END;
  START TRANSACTION;
  UPDATE accounts SET balance = balance - p_amount WHERE id = p_from;
  IF (SELECT balance FROM accounts WHERE id = p_from) < 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient funds';
  END IF;
  UPDATE accounts SET balance = balance + p_amount WHERE id = p_to;
  COMMIT;
END//
DELIMITER ;
CALL transfer(1, 2, 100.00);

-- PostgreSQL (11+)
CREATE PROCEDURE transfer(p_from int, p_to int, p_amount numeric)
LANGUAGE plpgsql AS $$
BEGIN
  UPDATE accounts SET balance = balance - p_amount WHERE id = p_from;
  IF (SELECT balance FROM accounts WHERE id = p_from) < 0 THEN
    RAISE EXCEPTION 'Insufficient funds';   -- whole CALL is rolled back
  END IF;
```

```
  UPDATE accounts SET balance = balance + p_amount WHERE id = p_to;
END $$;
CALL transfer(1, 2, 100);
```

- **Learn next:** `IN/OUT/INOUT` parameters, variables, loops, **cursors**, and error handling (MySQL `DECLARE … HANDLER`, `SIGNAL`, `GET DIAGNOSTICS`; PostgreSQL `BEGIN … EXCEPTION WHEN … END` blocks, `RAISE NOTICE` for debugging).
- **Function vs procedure in PostgreSQL:** a function runs inside the caller's transaction. A procedure can `COMMIT` in the middle, which makes it right for batch jobs such as "delete old rows 10,000 at a time, committing each batch".

2.6 Triggers

Why: to enforce rules or write an audit trail automatically. **Caution:** triggers are hidden logic. They slow every write and surprise developers, so document them and keep them small.

```
-- MySQL: an audit trigger
CREATE TABLE salary_audit (
  id BIGINT AUTO_INCREMENT PRIMARY KEY, emp_no INT,
  old_salary INT, new_salary INT,
  changed_by VARCHAR(100), changed_at DATETIME DEFAULT CURRENT_TIMESTAMP);
DELIMITER //
CREATE TRIGGER trg_salary_audit AFTER UPDATE ON salaries FOR EACH ROW
BEGIN
  IF NEW.salary <> OLD.salary THEN
    INSERT INTO salary_audit (emp_no, old_salary, new_salary, changed_by)
    VALUES (OLD.emp_no, OLD.salary, NEW.salary, CURRENT_USER());
  END IF;
END//
DELIMITER ;

-- PostgreSQL: a trigger function + a trigger
CREATE TABLE salary_audit (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY, emp_no int,
  old_salary int, new_salary int,
  changed_by text DEFAULT current_user, changed_at timestamptz DEFAULT now());
CREATE FUNCTION trg_salary_audit() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO salary_audit (emp_no, old_salary, new_salary)
  VALUES (OLD.emp_no, OLD.salary, NEW.salary);
  RETURN NEW;
END $$;
CREATE TRIGGER salary_audit AFTER UPDATE OF salary ON salaries
  FOR EACH ROW WHEN (OLD.salary IS DISTINCT FROM NEW.salary)
  EXECUTE FUNCTION trg_salary_audit();
```

Capability	MySQL	PostgreSQL
Row-level BEFORE/AFTER	Yes	Yes
Statement-level triggers	No	Yes (with transition tables `REFERENCING NEW TABLE AS …`)
Triggers on views	No	`INSTEAD OF`

Capability	MySQL	PostgreSQL
DDL triggers	No	Event triggers (`CREATE EVENT TRIGGER`)
Conditional trigger	`IF` inside the body	`WHEN (…)` clause

- **Replication note (MySQL):** with row-based replication, the trigger runs on the source and the replica applies only the resulting row changes. With statement-based replication, triggers run again on the replica.

2.7 Scheduling inside the database

- **MySQL Event Scheduler:** `event_scheduler=ON`, then `CREATE EVENT … ON SCHEDULE EVERY …`. On replicas, events are created as `DISABLED` on purpose, so jobs don't run twice.
- **PostgreSQL:** no built-in scheduler. Use the **`pg_cron`** extension (`SELECT cron.schedule('purge', '0 3 * * *', $$CALL purge_old()$$);`), or system cron/systemd timers.
- **Why it matters for HA:** after a failover, jobs must run on the *new* primary only. Plan for this. pg_cron runs in the database it was configured for, and on standbys it skips jobs.

2.8 Managing database code like a professional

- Keep all DDL and routines in git. Deploy with a migration tool (Flyway, Liquibase, Sqitch). Never edit production by hand.
- Know how to extract code: `SHOW CREATE PROCEDURE|TRIGGER|VIEW`, `information_schema.ROUTINES`; in PostgreSQL `\sf`, `\sv` and `pg_get_functiondef()`.
- Backups must include code: `mysqldump --routines --events --triggers`. In PostgreSQL routines are always in `pg_dump`, but roles are not (use `pg_dumpall --globals-only`).
- Review checklist for any routine: definer/owner account, `search_path`, privileges, transaction behaviour, performance under load, and what happens on a replica.

Lab project: build an "HR" mini-system on `employees`/`pagila` with a reporting view, a materialized summary, an audit trigger, a transfer-style procedure, a nightly purge job and least-privilege grants. Then build it again in the other engine.

Part 3 — Backup & restore

A backup is only proven by a restore. The professional standard is: physical backups + continuous log archiving (for point-in-time recovery) + regular logical dumps + an **automated restore test** — all scheduled, monitored and stored off the server.

3.1 Concepts first

- **RPO** (recovery point objective) = how much data you may lose. **RTO** (recovery time objective) = how long the restore may take. Every backup design starts from these two numbers, agreed with the business.
- **Logical vs physical:**
  - Logical backups (SQL/dump files) are portable across versions and allow single-table restores, but they are slow to restore at large sizes.

    - Physical backups (copies of data files) restore fast, but only to the same major version and platform.
- **Full / differential / incremental:** differential = changes since the last full; incremental = changes since the last backup of any type. Incrementals are smaller, but a restore needs every link in the chain.
- **Consistency:** a backup must be a single point in time. That is why `--single-transaction` exists, and why copying a running data directory with `cp` gives a corrupt backup.
- **PITR:** restore a base backup, then replay the binlog/WAL up to just before the mistake (for example, the `DROP TABLE` at 14:05).
- **3-2-1 rule:** 3 copies, on 2 types of media, 1 off-site. Add encryption, plus immutability against ransomware (object lock on S3).
- **Replication is not a backup:** a `DROP TABLE` replicates to every replica within milliseconds.

3.2 MySQL backup tools

Tool	Type	Use it for	Notes
`mysqldump`	Logical	Small DBs (< ~20 GB), single tables, migrations	`--single-transaction --source-data=2 --routines --events --triggers`. Restore is single-threaded.
MySQL Shell dump utilities	Logical, parallel	Medium/large logical backups, migrations to 8.4/9.7	`util.dumpInstance()`, `util.dumpSchemas()`, `util.loadDump()`. Chunked, compressed, parallel. Can write to S3-compatible storage.
`mydumper` / `myloader`	Logical, parallel	Open-source alternative to Shell dumps	Popular in large shops.
Percona XtraBackup	Physical, hot	The main production backup for InnoDB	Full + incremental, streaming (`xbstream`), compression, encryption, `xbcloud` to S3. Use the series that matches the server: 8.4 for 8.4, 9.7 for 9.7.
MySQL Enterprise Backup	Physical, hot	Oracle-licensed shops	Commercial. Similar to XtraBackup.
Clone plugin	Physical	Provisioning replicas and cluster members	`CLONE INSTANCE FROM ….` Not a retention tool.
Binlogs + `mysqlbinlog`	Log	PITR	Stream binlogs off the server continuously: `mysqlbinlog --read-`

Tool	Type	Use it for	Notes
			`from-remote-server --raw --stop-never`.

`mysqlpump` was removed in MySQL 8.4, so skip it even though many blog posts still use it.

XtraBackup lifecycle — learn every step and why it exists:

```
# 1. Backup (hot; copies data files + the redo written during the copy)
xtrabackup --backup --target-dir=/backup/full --parallel=4
# 2. Incremental (only pages changed since the base, found by LSN)
xtrabackup --backup --target-dir=/backup/inc1 --incremental-basedir=/backup/full
# 3. Prepare = crash recovery on the copy, which makes it consistent
xtrabackup --prepare --apply-log-only --target-dir=/backup/full
xtrabackup --prepare --target-dir=/backup/full --incremental-dir=/backup/inc1
# 4. Restore (server stopped, empty datadir)
xtrabackup --copy-back --target-dir=/backup/full
chown -R mysql:mysql /var/lib/mysql
```

`--apply-log-only` stops uncommitted transactions from being rolled back too early. Without it, later incrementals cannot be applied. Use it on every prepare step except the last.

MySQL PITR: restore the full backup → read the binlog position or GTID set from `xtrabackup_binlog_info` → replay binlogs up to just before the mistake:

```
mysqlbinlog --start-position=<pos> --stop-datetime="2026-10-31 14:04:59"
binlog.000042 binlog.000043 | mysql
# or skip only the bad transaction with GTIDs: --exclude-gtids='<uuid>:1234'
```

3.3 PostgreSQL backup tools

Tool	Type	Use it for	Notes
`pg_dump` / `pg_restore`	Logical, per database	Small DBs, single objects, version migrations	Use `-Fc` (custom) or `-Fd -j 4` (directory, parallel). `pg_restore -l` / `-L` restores only selected objects.
`pg_dumpall --globals-only`	Logical	Roles and tablespaces	`pg_dump` does **not** include roles. Always run this too.
`pg_basebackup`	Physical	Full cluster copy, building standbys	`-X stream --checkpoint=fast`. PG 17+ supports incrementals (`--incremental`, needs `summarize_wal=on`) merged with `pg_combinebackup`. Check with `pg_verifybackup`.
WAL archiving	Log	PITR	`archive_mode=on` + `archive_command`; restore with `restore_command` + `recovery_target_time`.

Tool	Type	Use it for	Notes
pgBackRest	Physical + WAL	**The usual production choice**	Full/diff/incr, parallel, compression, encryption, S3/Azure/GCS repos, async archiving, backup from a standby, `verify`. Works well with Patroni.
Barman	Physical + WAL	A central backup server for many clusters	From EDB. WAL streaming via `pg_receivewal`; `barman-cloud` for object storage.
WAL-G	Physical + WAL	Cloud and Kubernetes setups	Used by Spilo/Zalando-style deployments.

pgBackRest lifecycle:

```
# postgresql.conf
#   archive_mode = on
#   archive_command = 'pgbackrest --stanza=main archive-push %p'
pgbackrest --stanza=main stanza-create
pgbackrest --stanza=main check                  # proves archiving works end to end
pgbackrest --stanza=main --type=full backup
pgbackrest --stanza=main --type=diff backup
pgbackrest info
# PITR into an existing data directory (only changed files are copied)
pgbackrest --stanza=main --delta --type=time \
  --target="2026-10-31 14:04:59+02" --target-action=promote restore
```

Understand what restore does under the hood: it writes `recovery.signal` and `restore_command` into the configuration. PostgreSQL then fetches archived WAL and replays it until it reaches the target.

3.4 Choosing a strategy (a sensible default)

	MySQL	PostgreSQL
Weekly	XtraBackup full	pgBackRest full
Daily	XtraBackup incremental	pgBackRest diff (or incr)
Continuous	Binlog streaming to the backup host	WAL archiving (async)
Daily/weekly logical	MySQL Shell `dumpSchemas` (+ `mysqldump` for small DBs)	`pg_dump -Fd` + `pg_dumpall --globals-only`
Where	Backup host + S3-compatible object storage	pgBackRest repo on a backup host + S3
Take backups from	A replica (less load on the primary)	A standby (`backup-standby=y`)
Weekly test	Automatic restore to a scratch VM + checks	Automatic restore to a scratch VM + checks

3.5 Turning backups into reliable jobs

Why systemd timers rather than cron: output goes to journald, `Persistent=true` runs a missed job after a reboot, `OnFailure=` can trigger an alert, and `systemctl list-timers` shows the next run.

Every backup job script should:

1. take a lock (`flock`) so two runs never overlap;
2. use `set -euo pipefail` and return a clear exit code;
3. read credentials from a protected file, never from the command line;
4. check that the backup succeeded (XtraBackup prints `completed OK!`; use `pgbackrest info` / `verify`);
5. apply retention, copy off-site, and write a metric or status file that monitoring can read;
6. alert on failure **and** on "no successful backup in 26 hours".

```
#!/usr/bin/env bash
# /usr/local/bin/mysql-backup.sh  (full on Sunday, incremental on other days)
set -euo pipefail
exec 9>/var/lock/mysql-backup.lock; flock -n 9 || { echo "already running"; exit 1; }
BASE=/backup/mysql; TS=$(date +%F_%H%M); LOG=$BASE/logs/$TS.log
if [[ $(date +%u) -eq 7 || ! -L $BASE/last ]]; then
  TARGET=$BASE/full_$TS
  xtrabackup --defaults-extra-file=/etc/mysql-backup.cnf --backup --target-
dir="$TARGET" --parallel=4 2>"$LOG"
else
  TARGET=$BASE/inc_$TS
  xtrabackup --defaults-extra-file=/etc/mysql-backup.cnf --backup --target-
dir="$TARGET" \
    --incremental-basedir="$(readlink -f $BASE/last)" --parallel=4 2>"$LOG"
fi
grep -q "completed OK!" "$LOG"
ln -sfn "$TARGET" "$BASE/last"
find "$BASE" -maxdepth 1 -name 'full_*' -mtime +14 -exec rm -rf {} +   # retention
(also remove their incrementals)
date +%s > "$BASE/last_success"   # monitoring reads this
```

```
# /etc/systemd/system/mysql-backup.service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/mysql-backup.sh

# /etc/systemd/system/mysql-backup.timer
[Timer]
OnCalendar=*-*-* 01:30
Persistent=true
RandomizedDelaySec=10m
[Install]
WantedBy=timers.target
```

For PostgreSQL, pgBackRest already handles locking, retention (`repo1-retention-full=2`) and verification, so the timer simply runs `pgbackrest --stanza=main --type=diff backup`.

3.6 Backup jobs with Ansible

The same jobs, written as code so every server gets an identical, reviewed setup. Part 4 teaches the Ansible basics; these roles are the target to build towards.

```
# roles/mysql_backup/tasks/main.yml
- name: Install Percona XtraBackup 8.4
  ansible.builtin.dnf:
    name: percona-xtrabackup-84
    state: present

- name: Create the backup account with minimum privileges
  community.mysql.mysql_user:
    name: backup
    host: localhost
    password: "{{ vault_mysql_backup_password }}"
    priv: "*.*:BACKUP_ADMIN,PROCESS,RELOAD,LOCK TABLES,REPLICATION
CLIENT/performance_schema.log_status:SELECT"
    login_unix_socket: /var/lib/mysql/mysql.sock

- name: Deploy credentials file
  ansible.builtin.template:
    src: mysql-backup.cnf.j2
    dest: /etc/mysql-backup.cnf
    mode: "0600"

- name: Deploy backup script, service and timer
  ansible.builtin.template:
    src: "{{ item.src }}"
    dest: "{{ item.dest }}"
    mode: "{{ item.mode }}"
  loop:
    - { src: mysql-backup.sh.j2, dest: /usr/local/bin/mysql-backup.sh, mode: "0750" }
    - { src: mysql-backup.service.j2, dest: /etc/systemd/system/mysql-backup.service,
mode: "0644" }
    - { src: mysql-backup.timer.j2, dest: /etc/systemd/system/mysql-backup.timer,
mode: "0644" }

- name: Enable and start the timer
  ansible.builtin.systemd_service:
    name: mysql-backup.timer
    enabled: true
    state: started
    daemon_reload: true
```

```
# roles/pgbackrest/tasks/main.yml
- name: Install pgBackRest (PGDG repo)
  ansible.builtin.dnf:
    name: pgbackrest
    state: present

- name: Configure pgBackRest
  ansible.builtin.template:
    src: pgbackrest.conf.j2          # [global] repo1-path, repo1-retention-full,
```

```
[main] pg1-path
    dest: /etc/pgbackrest/pgbackrest.conf
    owner: postgres
    mode: "0640"

- name: Turn on WAL archiving
  become_user: postgres
  community.postgresql.postgresql_set:
    name: "{{ item.name }}"
    value: "{{ item.value }}"
  loop:
    - { name: archive_mode, value: "on" }       # needs a restart
    - { name: archive_command, value: "pgbackrest --stanza=main archive-push %p" }
  notify: restart postgresql

- name: Flush handlers so archiving is active before the stanza is created
  ansible.builtin.meta: flush_handlers

- name: Create the stanza (safe to re-run)
  become_user: postgres
  ansible.builtin.command: pgbackrest --stanza=main stanza-create
  changed_when: false

- name: Check archiving works end to end
  become_user: postgres
  ansible.builtin.command: pgbackrest --stanza=main check
  changed_when: false
```

With Patroni, set `archive_mode` and `archive_command` in the Patroni DCS configuration (`patronictl edit-config`) rather than with `postgresql_set`. Patroni owns the PostgreSQL configuration and would otherwise overwrite your change.

3.7 Restore labs (the part that makes you a professional)

1. Full restore of each tool to a new VM. **Time it** — that is your real RTO.
2. PITR: insert rows, note the time, `DROP TABLE`, then recover to one second before the drop. Do this in both engines.
3. Restore one table from a logical dump without touching the rest.
4. Rebuild a broken replica/standby from a backup (XtraBackup or pgBackRest), not from scratch.
5. Restore from off-site storage only, as if the backup host had burned down.
6. Corrupt a backup file on purpose, and confirm your verification step catches it.
7. Automate tests 1–2 as a weekly job that restores, runs checks (`CHECK TABLE`; the `amcheck` extension in PostgreSQL; row counts) and reports the result.

Part 4 — Ansible for database administration

The goal of this part is that every task from Parts 1–3 can be run by one reviewed, repeatable command. Rule of thumb: do a task by hand **once** to understand it, then write it in Ansible, then never do it by hand again.

Chapter 4.1 — Why Ansible, and the core ideas

- **Idempotency:** running a playbook twice gives the same result, and the second run reports `changed=0`. This is the most important idea. A playbook that reports changes on every run is hiding bugs.
- **Declarative state:** you describe the end state ("user `app_ro` exists with SELECT"), and the module works out the SQL.
- **Agentless:** it uses SSH + Python on the target. The modules run *on the database host*, so the database Python drivers must be installed there (PyMySQL for MySQL, psycopg2 or psycopg for PostgreSQL).
- **Code review and history:** every change goes through git, which gives you an audit trail for free.

Chapter 4.2 — Fundamentals (weeks 6–8, in parallel with Part 1)

1. **Inventory:** hosts, groups, `group_vars/`, `host_vars/`. Use one inventory per environment (lab, staging, prod).
2. **Ad-hoc commands:** `ansible dbservers -m ansible.builtin.ping`, `-m ansible.builtin.setup` (facts).
3. **Playbooks, plays, tasks and modules.** Always use full collection names (`ansible.builtin.template`).
4. **Variables and precedence:** learn the simplified order role defaults < group_vars < host_vars < play vars < `-e` extra vars.
5. **Templates (Jinja2):** generate `my.cnf` and `postgresql.conf` from variables, for example a buffer pool size calculated from `ansible_memtotal_mb`.
6. **Handlers:** restart or reload only when the config actually changed (`notify:`). Learn `meta: flush_handlers`.
7. **Loops, conditionals, `register`, `changed_when`, `failed_when`:** essential when you use `command`/`shell`, which are not idempotent on their own.
8. **Privilege escalation:** `become: true`, and `become_user: postgres` for PostgreSQL tasks.
9. **Check and diff mode:** `--check --diff` shows what *would* change. Run it before every production change.
10. **Tags, `--limit`, `serial`:** run part of a playbook, on part of the fleet, a few hosts at a time.

Chapter 4.3 — Project layout and collections

```
dba-ansible/
├── ansible.cfg
├── collections/requirements.yml     # community.mysql, community.postgresql,
ansible.posix
├── inventories/
│   ├── lab/hosts.yml
│   └── prod/hosts.yml
├── group_vars/
│   ├── all/vault.yml                 # encrypted with ansible-vault
│   ├── mysql/main.yml
│   └── postgres/main.yml
├── roles/
│   ├── os_prep/  mysql_install/  mysql_config/  mysql_users/  mysql_replication/
mysql_backup/
```

```
│   └── pg_install/  pg_config/  pg_hba/  pg_users/  etcd/  patroni/  pgbackrest/
pgbouncer/
└── playbooks/
    ├── site.yml  rolling_restart.yml  minor_upgrade.yml  add_user.yml
restore_test.yml
```

```
# inventories/lab/hosts.yml
all:
  children:
    mysql:
      hosts:
        my1: { ansible_host: 10.0.0.11, mysql_server_id: 1, mysql_role: source }
        my2: { ansible_host: 10.0.0.12, mysql_server_id: 2, mysql_role: replica }
    postgres:
      hosts:
        pg1: { ansible_host: 10.0.0.21 }
        pg2: { ansible_host: 10.0.0.22 }
        pg3: { ansible_host: 10.0.0.23 }
```

Key modules to learn:

Task	MySQL (`community.mysql`)	PostgreSQL (`community.postgresql`)
Databases	`mysql_db` (also dump/import)	`postgresql_db` (also dump/restore)
Users and roles	`mysql_user`, `mysql_role`	`postgresql_user`
Privileges	`priv:` in `mysql_user`/`mysql_role`	`postgresql_privs` (incl. `type: default_privs`)
Settings	`mysql_variables`	`postgresql_set`
Client access	`host` part of the account	`postgresql_pg_hba`
Replication	`mysql_replication`	`postgresql_slot`, `postgresql_publication`, `postgresql_subscription`
Arbitrary SQL	`mysql_query`	`postgresql_query`, `postgresql_script`
Extensions / plugins	`mysql_query` (`INSTALL COMPONENT`)	`postgresql_ext`
Inventory of a server	`mysql_info`	`postgresql_info`

Chapter 4.4 — Secrets with Ansible Vault

- `ansible-vault create group_vars/all/vault.yml`. Name vault variables `vault_*` and reference them from plain variables, so you can still `grep` for where a variable is used.
- Add `no_log: true` on any task that handles passwords, so they never appear in output or logs.
- Later: an external secrets store (HashiCorp Vault, a cloud secret manager) via lookup plugins.

Chapter 4.5 — MySQL roles

1. **`os_prep`:** sysctl, THP off (systemd unit), limits, chrony, firewall, SELinux labels, mounts.
2. **`mysql_install`:** add the MySQL repo, install the pinned version, enable the service. Read the temporary root password once, then switch root to socket authentication.
3. **`mysql_config`:** a template `my.cnf` with values per host (`server_id`) and calculated values (buffer pool), plus a restart handler.
4. **`mysql_secure`:** remove anonymous users, the `test` database and remote root (the equivalent of `mysql_secure_installation`, but repeatable).
5. **`mysql_users`:** roles and users defined as data in `group_vars`.
6. **`mysql_replication`:** configure replicas with GTID auto-positioning, then verify.
7. **`innodb_cluster`:** drive MySQL Shell (`mysqlsh --js -e "dba.createCluster(…)"`) behind a status check, so it is idempotent. Then bootstrap MySQL Router.

```
# roles/mysql_users/tasks/main.yml
- name: Create roles
  community.mysql.mysql_role:
    name: "{{ item.name }}"
    priv: "{{ item.priv }}"
    login_unix_socket: "{{ mysql_socket }}"
  loop: "{{ mysql_roles }}"      # e.g. { name: app_ro, priv: "sakila.*:SELECT" }

- name: Create users and give them their role
  community.mysql.mysql_user:
    name: "{{ item.name }}"
    host: "{{ item.host }}"
    password: "{{ item.password }}"
    login_unix_socket: "{{ mysql_socket }}"
  loop: "{{ mysql_app_users }}"
  no_log: true

- name: Grant roles and make them active at login
  community.mysql.mysql_query:
    login_unix_socket: "{{ mysql_socket }}"
    query:
      - "GRANT {{ item.role }} TO '{{ item.name }}'@'{{ item.host }}'"
      - "SET DEFAULT ROLE {{ item.role }} TO '{{ item.name }}'@'{{ item.host }}'"
  loop: "{{ mysql_app_users }}"
  no_log: true
```

```
# roles/mysql_replication/tasks/main.yml  (MySQL 8.4 syntax)
- name: Read replica status
  community.mysql.mysql_replication:
    mode: getreplica
    login_unix_socket: "{{ mysql_socket }}"
  register: repl

- name: Point the replica at the source (only when not configured yet)
  community.mysql.mysql_query:
    login_unix_socket: "{{ mysql_socket }}"
    query:
      - >-
```

```
        CHANGE REPLICATION SOURCE TO SOURCE_HOST='{{ mysql_source_host }}',
        SOURCE_USER='repl', SOURCE_PASSWORD='{{ vault_mysql_repl_password }}',
        SOURCE_AUTO_POSITION=1, SOURCE_SSL=1
      - START REPLICA
  when: not repl.is_replica
  no_log: true
```

Check the `community.mysql` documentation for your collection version: newer releases add MySQL 8.4 replication syntax to `mysql_replication` itself.

Chapter 4.6 — PostgreSQL and Patroni roles

```
# roles/pg_install/tasks/main.yml (Rocky + PGDG)
- name: Install PostgreSQL 18
  ansible.builtin.dnf:
    name: [postgresql18-server, postgresql18-contrib, python3-psycopg2]
    state: present

- name: Initialise the cluster once
  ansible.builtin.command: /usr/pgsql-18/bin/postgresql-18-setup initdb
  args:
    creates: /var/lib/pgsql/18/data/PG_VERSION    # makes the task idempotent

# roles/pg_hba/tasks/main.yml
- name: Allow the app network over TLS only
  community.postgresql.postgresql_pg_hba:
    dest: /var/lib/pgsql/18/data/pg_hba.conf
    contype: hostssl
    databases: appdb
    users: app_rw
    source: 10.0.0.0/24
    method: scram-sha-256
  notify: reload postgresql

# roles/pg_users/tasks/main.yml
- name: Read-only role can read future tables too
  become_user: postgres
  community.postgresql.postgresql_privs:
    login_db: appdb
    type: default_privs
    target_roles: app_owner
    schema: app
    objs: TABLES
    privs: SELECT
    roles: app_ro
```

- **`etcd` role:** a 3-member cluster with TLS, then check health with `etcdctl endpoint health`.
- **`patroni` role:** install Patroni, render `patroni.yml` from a template (scope, the etcd hosts, `bootstrap.dcs` with the PostgreSQL parameters, `pg_hba`, users). Start the first node, then the others, and wait until `patronictl list` shows one Leader and two streaming Replicas.
- **`haproxy` / `pgbouncer` roles:** health checks against Patroni's `/primary` and `/replica` endpoints on port 8008.
- **Reference project:** read **autobase** (formerly `postgresql_cluster`, by vitabaks). It is a complete Ansible deployment of Patroni + etcd + pgBackRest + PgBouncer + HAProxy. Study it to see how a mature project is structured; write your own roles first.

Chapter 4.7 — Day-2 operations playbooks

Day-2 operations are where Ansible pays off. Build these one by one:

1. **Rolling restart / config change:** `serial: 1`, replicas first, wait until each node is healthy before the next.
2. **Minor upgrade:** MySQL replicas → switchover → old primary. Patroni: upgrade replicas → `patronictl switchover` → upgrade the old leader.
3. **User provisioning:** a request becomes a pull request that adds an entry to `group_vars`, and the merge runs the playbook.
4. **Health report:** collect `mysql_info` / `postgresql_info`, replication status, disk use and the last backup time into one report.
5. **Restore test:** restore last night's backup on a scratch host, run checks, report, tear down.
6. **Failover drill:** a planned switchover playbook you run monthly.

```
# playbooks/rolling_restart_mysql.yml
- hosts: mysql
  serial: 1
  order: sorted          # put replicas before the source in your inventory
  become: true
  tasks:
    - name: Restart mysqld
      ansible.builtin.systemd_service:
        name: mysqld
        state: restarted

    - name: Wait for the port
      ansible.builtin.wait_for:
        port: 3306
        timeout: 300

    - name: Wait until replication is running again
      community.mysql.mysql_replication:
        mode: getreplica
        login_unix_socket: /var/lib/mysql/mysql.sock
      register: r
      until: r.Replica_IO_Running == 'Yes' and r.Replica_SQL_Running == 'Yes'
      retries: 30
      delay: 10
      when: mysql_role == 'replica'
```

Chapter 4.8 — Testing and quality

- **`ansible-lint`** on every commit.
- **Molecule** with the Podman or Docker driver: create a container, apply the role, run it a second time to check idempotency, run verify tests, then destroy. Every role should pass `molecule test`.
- **CI:** GitHub Actions or GitLab CI runs lint + Molecule on each pull request.
- **Staging first:** apply to the lab inventory, then staging, then prod with `--check --diff` before the real run.

Chapter 4.9 — Advanced

- Scheduling and access control for playbooks: AWX / Ansible Automation Platform, or Semaphore UI. This gives DBAs a button for "restore test" or "add user" with logs and permissions.
- Dynamic inventory from your cloud or CMDB.
- Execution environments (containers with pinned Ansible + collections) so every run uses the same versions.

Part 4 milestone: one command — `ansible-playbook -i inventories/lab playbooks/site.yml` — builds from empty VMs: a MySQL source + replica (or an InnoDB Cluster) and a 3-node Patroni cluster, with users, monitoring exporters, backups and timers. A second run reports `changed=0`.

Part 5 — Indexing & performance tuning

Most performance gains come from fixing queries and indexes, not from server parameters. Work in this order: **measure → find the top queries → read the plan → fix (query, index, schema) → verify → only then tune the server or hardware.**

5.1 How indexes really work (the why)

- **B-tree** is the default in both engines. It is a sorted, balanced tree, so it serves `=`, ranges, `ORDER BY` and prefix searches in O(log n).
- **InnoDB is a clustered index:** the table *is* the primary-key B-tree. Every secondary index stores the PK value, not a row pointer. Consequences:
    - a long PK makes every index bigger;
    - random PKs (UUIDv4) scatter inserts across the tree and hurt write speed and cache use. Prefer `BIGINT AUTO_INCREMENT` or time-ordered UUIDs (UUIDv7);
    - a secondary-index lookup is two lookups (secondary → PK) unless the index *covers* the query.
- **PostgreSQL tables are heaps**, and every index is secondary, pointing at a physical row location (`ctid`).
    - An **index-only scan** works only for pages marked all-visible in the visibility map, which `VACUUM` maintains. So vacuum health directly affects read speed.
    - **HOT updates** avoid touching indexes when no indexed column changes and the page has free space. Fewer indexes + a `fillfactor` below 100 on heavily updated tables = faster updates and less bloat.
- **Every index costs** extra work on INSERT/UPDATE/DELETE, disk, memory and vacuum time. An index has to earn its place.

5.2 Index types and when to use them

Type	Engine	Use for
B-tree	Both	Default: equality, ranges, sorting
Composite (multi-column)	Both	Queries filtering/sorting on several columns. Column order matters (5.3)

Type	Engine	Use for
Covering	Both	MySQL: add the columns to the index; PostgreSQL: `INCLUDE (…)`
Functional / expression	Both	`WHERE lower(email) = …` (MySQL 8.0.13+ functional key parts)
Partial	PostgreSQL	Index only some rows: `WHERE status = 'open'`
Invisible	MySQL	Test removing an index safely: `ALTER TABLE … ALTER INDEX i INVISIBLE`
Descending	Both	Mixed-direction sorts (`ORDER BY a ASC, b DESC`)
FULLTEXT	MySQL	Word search (`MATCH … AGAINST`)
Multi-valued	MySQL	JSON arrays
SPATIAL (R-tree)	MySQL	Geometry
GIN	PostgreSQL	`jsonb`, arrays, full-text search (`tsvector`), `LIKE '%x%'` with `pg_trgm`
GiST / SP-GiST	PostgreSQL	Geometry (PostGIS), ranges, nearest-neighbour, exclusion constraints
BRIN	PostgreSQL	Very large tables where values follow physical order (time-series logs). Tiny index
Hash	PostgreSQL	Equality only. Rarely better than B-tree

Both engines can **skip-scan** a composite index when the first column is not filtered (MySQL since 8.0.13; PostgreSQL B-tree skip scan since 18). Even so, design column order deliberately.

5.3 Designing indexes

1. **Start from the queries**, not the tables. Take the top queries from `pg_stat_statements` or the MySQL statement digests.
2. **Composite column order:** equality columns first, then the range column, then the sort columns. An index on `(a, b, c)` serves `a`, `a,b` and `a,b,c` (the **leftmost prefix** rule); it cannot help a filter on `b` alone, except through a skip scan.
3. **Selectivity:** a column with few distinct values (gender, a boolean) is rarely useful alone. In PostgreSQL, a partial index on the rare value can be.
4. **Keep predicates sargable** (able to use the index): avoid functions on the column (`WHERE date(created_at) = …` → use a range), implicit type casts (comparing a string column with a number), and leading wildcards (`LIKE '%abc'`).
5. **Index foreign keys in PostgreSQL.** MySQL creates them automatically; PostgreSQL does not. Without one, every `DELETE` on the parent scans the child table.
6. **Remove waste:**

    - unused indexes: `sys.schema_unused_indexes`; in PostgreSQL `pg_stat_user_indexes.idx_scan = 0`, checked on all replicas too;
    - duplicate or redundant indexes: `pt-duplicate-key-checker`, `sys.schema_redundant_indexes`; `(a)` is redundant when `(a, b)` exists.
7. **Build safely in production:** PostgreSQL `CREATE INDEX CONCURRENTLY` (and check for `INVALID` indexes if it fails); MySQL online DDL `ALGORITHM=INPLACE, LOCK=NONE`.

Worked example:

```
-- Query: open orders of one customer, newest first
SELECT id, total, created_at FROM orders
WHERE customer_id = 42 AND status = 'open'
ORDER BY created_at DESC LIMIT 20;

-- Good index: equality columns, then the sort column, then covered columns
-- MySQL
CREATE INDEX ix_orders_cust_status_created ON orders (customer_id, status, created_at
DESC, total);
-- PostgreSQL (covering + partial if 'open' is rare)
CREATE INDEX CONCURRENTLY ix_orders_open ON orders (customer_id, created_at DESC)
  INCLUDE (total) WHERE status = 'open';
```

5.4 Reading execution plans

- **MySQL:** `EXPLAIN` (look at `type`: `ALL` = full scan, `range`, `ref`, `eq_ref`, `const`; `key`; `rows` × `filtered`; and `Extra`: `Using filesort`, `Using temporary`, `Using index` = covering). `EXPLAIN FORMAT=TREE` shows the iterator tree. **`EXPLAIN ANALYZE`** runs the query and shows real times and row counts.
- **PostgreSQL:** `EXPLAIN (ANALYZE, BUFFERS)` (since 18, `ANALYZE` includes buffer counts automatically). Learn the node types: Seq Scan, Index Scan, Index Only Scan, Bitmap Heap Scan, Nested Loop, Hash Join, Merge Join, Sort, Aggregate.
- **The key skill — estimated vs actual rows:** when the planner expects 10 rows and gets 1,000,000, it picks the wrong join method. The cause is usually stale or insufficient statistics, or correlated columns. Fix the statistics before adding hints.
- **Buffers:** `shared hit` = served from memory, `read` = from disk. Huge reads for a small result usually mean a missing index.
- **Helpers:** paste PostgreSQL plans into explain.depesz.com or explain.dalibo.com.
- **Practise:** take 20 slow queries from `employees`/`pagila` under `sysbench`/`pgbench` load and write a one-line diagnosis for each plan before fixing it.

5.5 Statistics

- **MySQL:** `ANALYZE TABLE`, persistent statistics (`innodb_stats_persistent_sample_pages`), and **histograms** for non-indexed columns (`ANALYZE TABLE t UPDATE HISTOGRAM ON col WITH 64 BUCKETS`).
- **PostgreSQL:** `ANALYZE`, `default_statistics_target`, per-column `ALTER TABLE … ALTER COLUMN … SET STATISTICS 1000`, and **extended statistics** for correlated columns (`CREATE STATISTICS s (dependencies, ndistinct) ON city, zip FROM addresses`).

5.6 Rewriting queries

- Select only the columns you need. That makes covering indexes possible.
- **Keyset pagination** (`WHERE id > :last ORDER BY id LIMIT 50`) instead of large `OFFSET` values, which read and discard every skipped row.
- Replace `OR` across different columns with `UNION ALL` when each side can use its own index.
- Remove N+1 query patterns in the application. Batch inserts and updates (500–5,000 rows per transaction).
- Use `EXISTS` for "is there any"; use `LATERAL` (PostgreSQL) or window functions for "top N per group".
- Exact `COUNT(*)` on large tables is always expensive. Use estimates (`pg_class.reltuples`, `information_schema.TABLES`) or counter tables when an approximate number is acceptable.

5.7 Server tuning (after the queries)

Area	MySQL	PostgreSQL
Memory	`innodb_buffer_pool_size` (watch the hit ratio), `innodb_buffer_pool_instances`	`shared_buffers`, `effective_cache_size`, `work_mem` (× sort nodes × connections!), huge pages
Write path	`innodb_redo_log_capacity`, `innodb_flush_method=O_DIRECT`, `innodb_io_capacity/_max` for your disks	`max_wal_size`, `checkpoint_timeout`, `checkpoint_completion_target=0.9`, `wal_compression`
Storage cost model	—	`random_page_cost=1.1` on SSD, `effective_io_concurrency`, `io_method` (18)
Connections	Fewer, longer connections; thread pool (Percona Server / Enterprise); ProxySQL	**PgBouncer** in transaction mode; keep `max_connections` in the low hundreds
Background work	Purge threads; watch history list length	Autovacuum workers, cost limit, per-table scale factors
Durability trade-off	`innodb_flush_log_at_trx_commit=2`, `sync_binlog=0` only where losing ~1 s of data is acceptable	`synchronous_commit=off` per session or transaction, for unimportant writes

Starting-point calculators (MySQLTuner, PGTune) are fine for a first config. Then validate every change with a benchmark.

5.8 Partitioning & sharding

Partitioning splits one table into pieces **inside one server**. Sharding splits the data **across several servers**. Partition first; shard only when one primary can no longer handle the write load or the data size.

	Partitioning	Sharding
Where the pieces live	One server (and its replicas)	Many independent servers
Main reason	Data lifecycle (drop old months instantly), partition pruning	Write throughput and size beyond one machine
Application changes	None	Usually some: a shard key in queries, fewer cross-shard joins
Transactions and joins	Normal	Cross-shard ones are slower or limited
Complexity	Low	High: routing, rebalancing, backups and schema changes per shard
Built in?	Yes, both engines	No: Vitess (MySQL), Citus (PostgreSQL), or application-level

5.8.1 Partitioning — the rules first

- **Pick the partition key from the queries and the retention policy.** Usually it's a date (`created_at`) for time-based data, or a tenant/region for LIST partitioning.
- **Pruning only works when queries filter on the key.** Queries that don't filter on it must visit every partition and can get *slower*.
- **Keep the partition count moderate:** tens to a few hundred. Thousands of partitions slow down planning and use more memory.
- **Unique keys, including the PK, must contain the partition key** in both engines. That's why the PK below is `(id, created_at)`.
- **Automate creating future partitions and dropping old ones.** A missing future partition makes inserts fail (MySQL) or land in the default partition (PostgreSQL).

5.8.2 MySQL partitioning

```
CREATE TABLE orders (
  id          BIGINT NOT NULL AUTO_INCREMENT,
  customer_id INT NOT NULL,
  created_at  DATETIME NOT NULL,
  total       DECIMAL(12,2),
  PRIMARY KEY (id, created_at),            -- must include the partition key
  KEY ix_cust (customer_id, created_at)
)
PARTITION BY RANGE COLUMNS (created_at) (
```

```
  PARTITION p2026_10 VALUES LESS THAN ('2026-11-01'),
  PARTITION p2026_11 VALUES LESS THAN ('2026-12-01'),
  PARTITION p2026_12 VALUES LESS THAN ('2027-01-01'),
  PARTITION pmax     VALUES LESS THAN (MAXVALUE)
);

-- Add next month by splitting the catch-all partition
ALTER TABLE orders REORGANIZE PARTITION pmax INTO (
  PARTITION p2027_01 VALUES LESS THAN ('2027-02-01'),
  PARTITION pmax     VALUES LESS THAN (MAXVALUE));

-- Retention: instant, unlike DELETE
ALTER TABLE orders DROP PARTITION p2026_10;
-- Archive: swap a partition with an empty table of identical structure
ALTER TABLE orders EXCHANGE PARTITION p2026_11 WITH TABLE orders_archive_2026_11;

-- Check pruning and sizes
EXPLAIN SELECT * FROM orders WHERE created_at >= '2026-12-01';   -- see the
"partitions" column
SELECT PARTITION_NAME, TABLE_ROWS FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'orders';
```

- **Other types:** `LIST COLUMNS (region)`; `HASH (customer_id) PARTITIONS 8` or `KEY` to spread rows evenly. You can also sub-partition.
- **Limitations to know:** partitioned InnoDB tables **cannot have foreign keys**, either referencing or referenced. Every unique key must include all partition columns. All partitions use the same storage engine.
- **Converting an existing large table:** use `gh-ost` or `pt-online-schema-change` with the `PARTITION BY` clause, so the table isn't locked for hours.
- **Automation:** a monthly MySQL event, or an Ansible task, that runs the `REORGANIZE` and `DROP` statements.

5.8.3 PostgreSQL declarative partitioning

```
CREATE TABLE orders (
  id          bigint GENERATED ALWAYS AS IDENTITY,
  customer_id int NOT NULL,
  created_at  timestamptz NOT NULL,
  total       numeric(12,2),
  PRIMARY KEY (id, created_at)             -- must include the partition key
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2026_11 PARTITION OF orders
  FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');
CREATE TABLE orders_2026_12 PARTITION OF orders
  FOR VALUES FROM ('2026-12-01') TO ('2027-01-01');
CREATE TABLE orders_default PARTITION OF orders DEFAULT;    -- safety net; keep it
empty

CREATE INDEX ON orders (customer_id, created_at);            -- created on every
partition

-- Retention without blocking queries on the parent
ALTER TABLE orders DETACH PARTITION orders_2026_11 CONCURRENTLY;
```

```
DROP TABLE orders_2026_11;          -- or pg_dump it to archive storage first

-- Bulk-load a month offline, then attach it with minimal locking
CREATE TABLE orders_2027_01 (LIKE orders INCLUDING DEFAULTS);
-- … load data …
ALTER TABLE orders_2027_01 ADD CONSTRAINT chk_range
  CHECK (created_at >= '2027-01-01' AND created_at < '2027-02-01');  -- lets ATTACH
skip its validation scan
ALTER TABLE orders ATTACH PARTITION orders_2027_01
  FOR VALUES FROM ('2027-01-01') TO ('2027-02-01');

EXPLAIN SELECT * FROM orders WHERE created_at >= '2026-12-15';    -- only matching
partitions appear
```

- **Other types:** `PARTITION BY LIST (region)`; `PARTITION BY HASH (customer_id)` with `FOR VALUES WITH (MODULUS 4, REMAINDER 0)`. Partitions can themselves be partitioned (for example by month, then by hash).
- **Foreign keys work** to and from partitioned tables. That's a big difference from MySQL.
- **Why keep the DEFAULT partition empty:** when you add a new partition, PostgreSQL must scan the default partition to check that no rows belong in it, and it fails if any do.
- **Automate with `pg_partman`:**

```
CREATE SCHEMA partman;
CREATE EXTENSION pg_partman SCHEMA partman;
SELECT partman.create_parent(
  p_parent_table => 'public.orders', p_control => 'created_at',
  p_interval => '1 month', p_premake => 3);                -- always 3 future months
ready
UPDATE partman.part_config
   SET retention = '12 months', retention_keep_table = false
 WHERE parent_table = 'public.orders';
SELECT cron.schedule('partman-maint', '15 * * * *',
  $$CALL partman.run_maintenance_proc()$$);                -- via pg_cron
```

- **Converting an existing large table:** create the new partitioned table, then either attach the old table as one big historical partition (with a matching `CHECK` constraint) and route new data to new partitions, or copy in batches and swap names in one short transaction. Logical replication is another option.

5.8.4 Sharding — concepts before tools

Try these first, because each is far cheaper than sharding: query and index fixes, a bigger server, read replicas for reads, caching, partitioning, and splitting unrelated data into separate databases (a functional split).

- **The shard key is the most important decision.** It should have many distinct values, spread load evenly, and appear in almost every query. Typical choices are `tenant_id` or `customer_id`. Changing it later means moving all the data.
- **Distribution strategies:**
  - **hash** — even spread, but range scans hit every shard;
  - **range** — good for range queries, but risks "hot" shards;
  - **directory/lookup** — a mapping table says where each tenant lives; the most flexible option, but one more component to keep highly available.

- **Co-location:** keep related rows (a customer, their orders, their payments) on the same shard, so joins and transactions stay on one shard.
- **Cross-shard work** (queries, joins, transactions) is slower and needs two-phase commit for atomicity. Design so that 95%+ of queries hit one shard.
- **Global unique IDs:** each shard can't use its own `AUTO_INCREMENT`/sequence without clashes. Use UUIDv7, a Snowflake-style ID, or the sequence features of the sharding tool.
- **Day-2 operations get harder:**
    - resharding and rebalancing when shards grow;
    - a consistent backup across shards;
    - schema changes on every shard;
    - monitoring N clusters instead of one.

5.8.5 MySQL sharding options

Option	How it works	When to use it
Vitess	`vtgate` routes queries using a VSchema; `vttablet` manages each MySQL shard; resharding workflows move data online	The main open-source choice for large MySQL sharding. CNCF graduated; used by YouTube, Slack, GitHub; PlanetScale is hosted Vitess
Application-level	The application looks up the shard in a shard map and connects there	Small number of shards, simple tenant-per-shard designs
ProxySQL query rules	Routes by schema name or query pattern to different hostgroups	Simple routing by tenant/schema, without automatic resharding
MySQL NDB Cluster	A different storage engine that shards automatically across data nodes	Special low-latency, in-memory-style workloads. Not a drop-in for InnoDB

Vitess essentials — a VSchema says how a table is sharded:

```
{
  "sharded": true,
  "vindexes": { "hash": { "type": "hash" } },
  "tables": {
    "customers": { "column_vindexes": [ { "column": "customer_id", "name": "hash" } ]
},
    "orders":    { "column_vindexes": [ { "column": "customer_id", "name": "hash" } ]
}
  }
}
```

```
# Split one unsharded shard "0" into two shards online, then cut traffic over
vtctldclient Reshard --workflow cust2x --target-keyspace commerce create \
  --source-shards '0' --target-shards '-80,80-'
vtctldclient Reshard --workflow cust2x --target-keyspace commerce show
vtctldclient Reshard --workflow cust2x --target-keyspace commerce switchtraffic
vtctldclient Reshard --workflow cust2x --target-keyspace commerce complete
```

The shard names `-80` and `80-` are ranges of the hashed key space (the lower and upper halves). Start with the Vitess "local" example or the Vitess Kubernetes operator, and check the release notes for which MySQL versions your Vitess release supports.

5.8.6 PostgreSQL sharding options

Option	How it works	When to use it
Citus	An open-source extension. A coordinator node routes queries to worker nodes; tables are distributed by a column	The main choice for sharding PostgreSQL: multi-tenant SaaS and real-time analytics
`postgres_fdw` + partitioning	Each partition is a foreign table on another server	Learning, or simple DIY setups. Limited pushdown and no distributed planner
Application-level	The application chooses the shard	Tenant-per-database designs
Distributed SQL databases (YugabyteDB, CockroachDB)	PostgreSQL-compatible databases that shard natively	New systems that need global distribution. They are not PostgreSQL itself

Citus essentials:

```
-- On the coordinator (the citus extension is also installed on every worker)
CREATE EXTENSION citus;
SELECT citus_add_node('pg-worker1', 5432);
SELECT citus_add_node('pg-worker2', 5432);

CREATE TABLE customers (customer_id bigint PRIMARY KEY, name text);
CREATE TABLE orders (
  customer_id bigint, id bigint, created_at timestamptz, total numeric,
  PRIMARY KEY (customer_id, id));                 -- distribution column must be in
the PK
CREATE TABLE countries (code text PRIMARY KEY, name text);

SELECT create_distributed_table('customers', 'customer_id');
SELECT create_distributed_table('orders', 'customer_id', colocate_with =>
'customers');
SELECT create_reference_table('countries');      -- small table copied to every node

SELECT citus_rebalance_start();                  -- after adding a worker, move
shards online
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;   -- routed to a single shard
```

- Citus can distribute time-partitioned tables, so you can combine both techniques.
- Each Citus node is a normal PostgreSQL server, so you give each one HA the way you already know (Patroni supports Citus clusters) and back each one up with pgBackRest.
- Citus support for a new PostgreSQL major usually arrives some months after that major's release. Check the Citus release notes before you upgrade.

DIY sharding with `postgres_fdw` (good for understanding how sharding works under the hood):

```
CREATE EXTENSION postgres_fdw;
CREATE SERVER shard1 FOREIGN DATA WRAPPER postgres_fdw OPTIONS (host 'pg-shard1',
dbname 'app');
CREATE SERVER shard2 FOREIGN DATA WRAPPER postgres_fdw OPTIONS (host 'pg-shard2',
dbname 'app');
CREATE USER MAPPING FOR app_rw SERVER shard1 OPTIONS (user 'app_rw', password '***');
CREATE USER MAPPING FOR app_rw SERVER shard2 OPTIONS (user 'app_rw', password '***');

CREATE TABLE orders (customer_id bigint, id bigint, total numeric) PARTITION BY HASH
(customer_id);
-- The matching tables must already exist on each shard
CREATE FOREIGN TABLE orders_p0 PARTITION OF orders FOR VALUES WITH (MODULUS 2,
REMAINDER 0) SERVER shard1;
CREATE FOREIGN TABLE orders_p1 PARTITION OF orders FOR VALUES WITH (MODULUS 2,
REMAINDER 1) SERVER shard2;
```

5.8.7 Partitioning & sharding labs

1. Partition a 50-million-row `orders` table by month in both engines. Compare `DELETE` of one month with `DROP`/`DETACH` of a partition (time taken, lock waits, bloat or undo growth).
2. Show pruning and its absence: run the same query with and without the partition key in `WHERE` and compare the plans.
3. Automate partition maintenance: a MySQL event or Ansible task for MySQL, and `pg_partman` + `pg_cron` for PostgreSQL. Then let the lab run past a month boundary and check that it worked.
4. Build a Citus cluster (1 coordinator + 2 workers, Docker Compose or 3 VMs). Distribute `customers`/`orders`, add a third worker, rebalance, and compare single-shard with cross-shard query plans.
5. Build the Vitess local example. Start unsharded, then reshard into `-80,80-` while a load generator is running, and confirm that no writes were lost.
6. Write down for each lab: the shard or partition key you chose, why, and which queries became slower because of it.

5.9 Benchmarking and performance monitoring

- **Benchmark tools:** `sysbench` (MySQL, and PostgreSQL too), `pgbench`. Rules: one change at a time, a warmed-up cache, runs of at least 10–15 minutes, report p95/p99 latency, not only throughput.
- **Always-on query monitoring:**
    - PostgreSQL: `pg_stat_statements` (sort by `total_exec_time`), `auto_explain` (logs plans of slow queries);
    - MySQL: `performance_schema` statement digests, `sys.statements_with_runtimes_in_95th_percentile`, slow log + `pt-query-digest`;
    - both: PMM Query Analytics.
- **Replication lag tuning:** a multi-threaded applier in MySQL; in PostgreSQL, look for long queries on standbys (`hot_standby_feedback`, `max_standby_streaming_delay` trade-offs).

5.10 Performance labs

1. Load 10 million rows. Measure a query without an index, with a single-column index, with a composite index and with a covering index. Compare plans, buffers and time.
2. Reproduce a wrong plan caused by correlated columns, then fix it with extended statistics (PostgreSQL) or a histogram (MySQL).
3. Show write slowdown: measure insert throughput with 1, 5 and 10 indexes on a table.
4. Compare UUIDv4 and BIGINT/UUIDv7 primary keys for insert speed in InnoDB.
5. Tune a deliberately bad config (tiny buffer pool / `shared_buffers`, tiny redo/WAL) under `sysbench`/`pgbench` and record the before/after numbers.
6. Put PgBouncer in front of PostgreSQL with 1,000 client connections and compare latency with and without it.

Video courses for every part

If reading documentation is hard going, use one video course per part as your main teacher, and use this roadmap as the lab checklist. A practical shortlist: Hussein Nasser for the "why", Imran Afzal for MySQL, Sadiq Basha for PostgreSQL, Jeff Geerling (free) for Ansible, and Dan Sullivan + Lucian Oprea for performance.

How to learn from videos without losing the "why": watch one section → close the video and do the matching lab here on your own VMs → explain it out loud or in your notes in your own words. If you can't explain it, rewatch the section. Udemy prices drop to about $10–20 during its frequent sales, and Udemy refunds within 30 days. Ratings below are from the course listing pages as of 2026.

Foundations — the "why" behind everything (start here, in parallel with Part 1)

Course	Platform	Why this one
Fundamentals of Database Engineering — Hussein Nasser	Udemy, ~24 h, 4.7 (9,000+ ratings)	**Core pick.** Explains ACID, isolation, indexing, B-trees, partitioning, replication and sharding by showing *why* each exists, using PostgreSQL and MySQL. It is the closest match to this roadmap's "why" approach.
CMU 15-445 Intro to Database Systems — Andy Pavlo	YouTube, free, 26 lectures	University-level internals: storage, buffer pools, indexes, MVCC, logging and recovery. Watch the lectures on buffer pools, tree indexes, MVCC, logging and recovery; skip the C++ projects.

Part 1 — Administration

Course	Platform	Why this one
Complete MySQL Database Administration Course — Imran Afzal, Abbas Mehmood	Udemy, ~28 h, 4.6 (~2,000 ratings),	**Core MySQL pick.** Install, configure, manage and troubleshoot MySQL on Linux, at a slow, clear pace.

Course	Platform	Why this one
	updated 2026	
Becoming a Production MySQL DBA	Udemy, ~20 h, 300 lectures	Production view: accounts and roles, GTID replication, upgrades, Docker, troubleshooting, with DBA interview questions. A good second MySQL course.
MySQL Replication course from zero to hero — Mughees Ahmed	Udemy, ~7.5 h, 4.3	Replication topologies and building replicas with mysqldump, XtraBackup, mydumper and the clone plugin. Last updated 2022, so translate MASTER/SLAVE commands to the 8.4 syntax in 1.8.
PostgreSQL Database Administration on Windows/Linux — Sadiq Basha (Parts 1, 2, 3)	Udemy; Part 2 alone ~8 h, 4.5 (~1,900 ratings)	**Core PostgreSQL pick.** Part 1 = fundamentals; Part 2 = log shipping, streaming and synchronous replication, slots, failover, logical replication; Part 3 = performance tuning. It uses repmgr for HA; the concepts carry over directly to Patroni.
Postgres Cluster: Master Patroni-Managed Clusters — Sandro Servino	Udemy, ~12 h	Patroni + etcd specifically, to revisit your 3-node build with explanations. Only a handful of reviews so far, so watch the free preview first.
MySQL InnoDB Cluster in a Nutshell, Part 1 and Part 2 — Kenny Gryp, Frédéric Descamps	Percona, free tutorial	How Group Replication works inside, and migrating source/replica setups to InnoDB Cluster, taught by the people who built and promoted it.

If your employer pays for instructor-led training: CYBERTEC — PostgreSQL HA with Patroni (3 days online), Percona MySQL training (including Group Replication 101), or EDB Foundations of PostgreSQL ($1,200, includes a certification path).

Part 2 — Views, functions, procedures, triggers

Course	Platform	Why this one
SQL and PostgreSQL: The Complete Developer's Guide — Stephen Grider	Udemy, ~22 h, 4.7 (11,000+ ratings)	Very clear teacher. Covers schema design, complex queries, query tuning, working with millions of rows, migrations and transactions. The SQL ideas apply to MySQL too.
Advanced SQL: Window Functions — Ami Levin	LinkedIn Learning	Window functions explained properly (2.1).

There is no outstanding video course dedicated to MySQL stored programs. Use the stored-routine sections of the MySQL DBA course above, then build the Part 2 lab project in both engines.

Part 3 — Backup & restore

No single course covers modern backup tooling well, so combine these:

- The XtraBackup, mydumper and clone sections of the **MySQL Replication zero-to-hero** course above.
- The backup and PITR material in **Sadiq Basha's PostgreSQL series** above.
- Postgres FM — the pgBackRest episode with David Steele, pgBackRest's author (free; also on YouTube).
- Then do the 3.7 restore labs. Restores are learned by doing them, not by watching.

Part 4 — Ansible

Course	Platform	Why this one
Ansible 101 — Jeff Geerling	YouTube, free, 15 × ~1 h	**Core pick.** By the author of *Ansible for DevOps*: inventory, playbooks, handlers, variables, Vault, roles, Molecule testing, CI, AWX and real-world playbooks — exactly the chapters in Part 4.
Ansible for the Absolute Beginner — Mumshad Mannambeth (KodeKloud)	Udemy, short, 4.5 (45,000+ ratings)	A gentle start with labs that run in the browser. Take it first if Ansible is completely new to you.
Complete DevOps Ansible Automation Training — Imran Afzal	Udemy, 4.7 (~4,000 ratings)	Linux-admin focused and aimed at RHCE. Take it if you plan to sit the RHCE exam.

No course teaches "Ansible for DBAs" end to end. After Ansible 101, write the Part 4 roles yourself, and read the autobase project (4.6) as a worked example.

Part 5 — Indexing, performance, partitioning & sharding

Course	Platform	Why this one
Advanced SQL for Query Tuning and Performance Optimization — Dan Sullivan	LinkedIn Learning, ~2 h, updated 2023	**Core pick.** Short and focused: execution plans, index types, join methods, partitioning and materialized views, using PostgreSQL.
PostgreSQL High Performance Tuning Guide — Lucian Oprea	Udemy, 4.2 (~1,600 ratings), updated 2026	Server and query tuning, monitoring and preventing performance problems in PostgreSQL.
Postgres Performance Demystified + the weekly *Scaling*	Free	A free course plus 15-minute weekly episodes on real performance findings.

Course	Platform	Why this one
Postgres show — Creston Jamison		
Hussein Nasser's course (above)	Udemy	Its indexing, partitioning and sharding sections are the best video explanation for 5.1–5.3 and 5.8.

For MySQL-specific performance, the Hussein Nasser course and the MySQL DBA courses above cover the essentials. After that, Percona's free webinars and blog on their YouTube channel are the best ongoing source.

Keep learning every week (free)

- **Postgres FM** — a weekly podcast/video by Nikolay Samokhvalov and Michael Christofides, ~30 minutes per topic.
- **Scaling Postgres** — a weekly 15-minute roundup of PostgreSQL performance news.
- **Hussein Nasser's YouTube channel** — deep dives such as MySQL 8 locking.
- **Percona's YouTube channel** — MySQL and PostgreSQL webinars and conference talks.

Access tip: LinkedIn Learning is often included with LinkedIn Premium, and many employers and universities provide Udemy Business or LinkedIn Learning for free. Check before you buy.

Part 6 — Capstone, certifications and reading

The capstone proves the whole roadmap. Put it in a public git repo, because that repo is your portfolio for DBA interviews.

Capstone project

- ☐ Ansible builds, from empty VMs: a MySQL 8.4 InnoDB Cluster (3 nodes + Router) and a PostgreSQL 18 Patroni cluster (3 nodes + etcd + HAProxy + PgBouncer)
- ☐ Least-privilege roles and users, defined as data, with secrets in Vault
- ☐ Backups: XtraBackup + binlog streaming; pgBackRest + WAL archiving to S3-compatible storage (MinIO in the lab)
- ☐ Weekly automated restore test with a pass/fail report
- ☐ Monitoring: PMM or Prometheus + Grafana, with alerts for lag, disk, connections, backup age and transaction ID age
- ☐ Runbooks: failover, PITR, adding a node, minor and major upgrades
- ☐ A performance report: the top 10 queries before and after index and query fixes, with plans and numbers
- ☐ A major-upgrade rehearsal: MySQL 8.4 → 9.7 and PostgreSQL 18 → 19, each with a rollback plan

Certifications (optional, but they help with CVs)

- **MySQL:** Oracle's MySQL Database Administrator certification. Check Oracle University for the current exam code and version.

- **PostgreSQL:** EDB PostgreSQL certifications (associate/professional level); PostgreSQL CE (SRA OSS).
- **Ansible:** Red Hat Certified Engineer (RHCE, EX294) is an Ansible automation exam.

Recommended reading (the "why" sources)

- The Internals of PostgreSQL (free online) — the best explanation of MVCC, WAL, vacuum and the buffer manager.
- Use The Index, Luke (free online) — indexing for both engines.
- *Efficient MySQL Performance* (Daniel Nichter) and *High Performance MySQL*, 4th ed.
- *PostgreSQL Query Optimization* (Dombrovskaya, Novikov, Bailliekova) and *The Art of PostgreSQL* (Dimitri Fontaine).
- *Ansible for DevOps* (Jeff Geerling).
- Official guides: MySQL 8.4 manual, PostgreSQL docs, Patroni docs, pgBackRest user guide, Percona XtraBackup docs, community.mysql and community.postgresql collection docs.

Sources for version facts

- PostgreSQL 19 RC 1 announcement (GA planned 19 Nov 2026) and postgresql.org (18.6 current; PostgreSQL 14 end of fixes 12 Nov 2026)
- MySQL 9.7 LTS announcement blog and endoflife.date MySQL (9.7.4, 8.4.13, 8.0 end of life Apr 2026)
- Percona XtraBackup 9.7.1-rc1 release and Percona MySQL upgrade paths (LTS series cannot be skipped)

Appendix A — SQL Server for MySQL & PostgreSQL DBAs

About 70–80% of your MySQL/PostgreSQL knowledge carries over to SQL Server: the transaction log, backups + point-in-time recovery, clustered indexes, execution plans, statistics, locking and HA concepts. What you must learn new is the tooling (T-SQL, SSMS, PowerShell), the Windows/Active Directory platform, a few behaviours that differ by default, and licensing. Plan two weeks (weeks 48–49) for a first pass, and more if a job needs it.

A.1 Map of the roadmap to SQL Server

Roadmap topic	MySQL / PostgreSQL	SQL Server equivalent
Instance and databases (1.1)	One server / cluster holds many databases	One **instance** (default or named) holds many databases; system DBs `master`, `model`, `msdb`, `tempdb`
Crash log (1.1)	Redo log / WAL	Transaction log (`.ldf`), write-ahead logging, checkpoints
Config (1.3)	`my.cnf`, `postgresql.conf`	`sp_configure`, database options, `mssql-conf` on Linux, SQL Server Configuration Manager on Windows

Roadmap topic	MySQL / PostgreSQL	SQL Server equivalent
Accounts (1.4)	`user@host`, roles, `pg_hba.conf`	Server **logins** (Windows/AD or SQL auth) mapped to database **users**; server and database roles
Isolation (1.5)	MVCC, so readers don't block writers	Locking READ COMMITTED by default; turn on **RCSI** (read committed snapshot) for MVCC-like behaviour
Monitoring (1.6)	performance_schema, `pg_stat_*`	DMVs (`sys.dm_*`), Query Store, Extended Events, wait statistics
Scheduler (2.7)	Event Scheduler, pg_cron	**SQL Server Agent** (jobs, schedules, alerts; history in `msdb`)
Backup (3)	XtraBackup, pgBackRest + binlog/WAL	Built-in `BACKUP DATABASE` / `BACKUP LOG`; recovery models
HA (1.8)	Group Replication / InnoDB Cluster, Patroni + etcd + HAProxy	**Always On Availability Groups** on Windows Server Failover Clustering (Pacemaker on Linux) + listener
DR site (1.8)	ClusterSet, cascading standbys	Distributed Availability Groups, log shipping
Automation (4)	Ansible collections	**dbatools** (PowerShell), Ansible (`microsoft.sql`, `community.general.mssql_*`), Agent jobs
Partitioning (5.8)	Declarative partitioning	Partition function + partition scheme, `SWITCH`
Sharding (5.8)	Vitess, Citus	Nothing built in: application-level sharding, or Azure SQL elastic tools

A.2 Versions, editions, licensing and lab

- **Current version:** SQL Server 2025 (version 17.x), generally available since 18 Nov 2025. It runs on Windows, Linux (RHEL 10 and Ubuntu 24.04 supported from CU1) and containers.
- **Editions:** Enterprise, Standard, Express (free, size-limited) and **Developer** (free, all features, non-production only). The Web edition was discontinued from 2025.
- **Licensing is part of the DBA job:** production is licensed per CPU core, and many features depend on the edition (for example, Standard only has a limited "Basic" availability group with 2 replicas and 1 database). Always check the edition before you design a solution.
- **Patching:** Cumulative Updates (CUs) are released regularly. Test them like minor versions in MySQL/PostgreSQL.
- **Lab:**
    a. SQL Server 2025 Developer in Docker on one of your Rocky/Ubuntu VMs (image `mcr.microsoft.com/mssql/server`).

b. A Windows Server evaluation VM with Active Directory, for the Windows-only parts: AD logins and Windows clustering.
c. Tools: SSMS on Windows, `sqlcmd` everywhere, PowerShell + dbatools.

A.3 Architecture, installation and configuration

- Each database has data files (`.mdf` / `.ndf`) and a transaction log (`.ldf`). Pages are 8 KB, grouped in extents. The buffer pool caches pages; the lazy writer and checkpoints write them.
- **`tempdb`** is shared by the whole instance (temp tables, sorts, row versions for RCSI). Give it several equally sized data files on fast disk. It is a classic bottleneck.
- **First settings to set after installing:**

```
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'max server memory (MB)', 26000;      -- leave RAM for the OS
(here: 32 GB host)
EXEC sp_configure 'max degree of parallelism', 8;      -- often = cores per NUMA
node, max 8
EXEC sp_configure 'cost threshold for parallelism', 50; -- the default of 5 is too
low
EXEC sp_configure 'backup compression default', 1;
EXEC sp_configure 'optimize for ad hoc workloads', 1;
RECONFIGURE;
ALTER DATABASE Sales SET READ_COMMITTED_SNAPSHOT ON;  -- MVCC-like reads (test the
app first)
```

- On Windows, also grant the service account **Instant File Initialization** (faster file growth and restores) and consider **Lock Pages in Memory**.
- **Why it matters:** with the defaults, SQL Server takes almost all the RAM it can, and parallelism kicks in for tiny queries.

A.4 Security

- **Two levels:** a *login* lets you connect to the instance; a *user* inside each database gives access to that database. Restoring a database onto another server leaves **orphaned users** (a user whose login doesn't exist there). Fix them with `ALTER USER … WITH LOGIN = …`, or use contained database users.
- **Windows/AD authentication** is preferred: no passwords in connection strings, and access is managed through AD groups.
- **Roles:** fixed server roles (`sysadmin` = root/superuser), fixed database roles (`db_datareader`, `db_datawriter`, `db_owner`), and your own roles. Grant on a **schema** to cover future tables automatically. In PostgreSQL that needs `ALTER DEFAULT PRIVILEGES`.

```
CREATE LOGIN app_ro WITH PASSWORD = 'Use-A-Vault-Secret-1!', CHECK_POLICY = ON;
USE Sales;
CREATE USER app_ro FOR LOGIN app_ro;
CREATE ROLE reporting;
GRANT SELECT ON SCHEMA::dbo TO reporting;   -- includes tables created later
ALTER ROLE reporting ADD MEMBER app_ro;
```

- **Built-in protection features:** TDE (encryption at rest), Always Encrypted (column encryption the server can't read), Row-Level Security, Dynamic Data Masking, and SQL Server Audit.

A.5 T-SQL programming objects (Part 2 equivalents)

- **Views:** normal views, and **indexed views** (SQL Server's materialized views: `WITH SCHEMABINDING` + a unique clustered index, kept up to date automatically).
- **Functions:**
  - *inline table-valued* functions are fast; they act like a parameterised view;
  - multi-statement and scalar functions were historically slow (scalar function inlining since 2019 helps).
- **Procedures** with `TRY/CATCH`, `THROW` and `SET XACT_ABORT ON`:

```
CREATE OR ALTER PROCEDURE dbo.transfer @from int, @to int, @amount decimal(12,2)
AS
BEGIN
  SET NOCOUNT ON; SET XACT_ABORT ON;      -- any error rolls back the whole
transaction
  BEGIN TRY
    BEGIN TRAN;
      UPDATE dbo.accounts SET balance -= @amount WHERE id = @from;
      IF (SELECT balance FROM dbo.accounts WHERE id = @from) < 0
        THROW 50001, 'Insufficient funds', 1;
      UPDATE dbo.accounts SET balance += @amount WHERE id = @to;
    COMMIT;
  END TRY
  BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;
    THROW;                                  -- pass the original error to the caller
  END CATCH
END;
```

- **Triggers fire once per statement, not once per row.** Changed rows are in the `inserted` and `deleted` pseudo-tables, so always write triggers for many rows:

```
CREATE OR ALTER TRIGGER dbo.trg_salary_audit ON dbo.salaries AFTER UPDATE AS
BEGIN
  SET NOCOUNT ON;
  INSERT dbo.salary_audit (emp_no, old_salary, new_salary, changed_by)
  SELECT d.emp_no, d.salary, i.salary, SUSER_SNAME()
  FROM inserted i
  JOIN deleted d ON d.emp_no = i.emp_no AND d.from_date = i.from_date
  WHERE i.salary <> d.salary;
END;
```

- Also: `INSTEAD OF` triggers on views, DDL triggers, logon triggers, sequences and `IDENTITY` columns. **SQL Server Agent** jobs replace cron/pg_cron.

A.6 Backup & restore

- **Recovery models decide what the log keeps:**
  - `SIMPLE` — the log is reused after each checkpoint, so no point-in-time recovery is possible.
  - `FULL` — point-in-time recovery is possible, but the log **only shrinks after log backups**. Forgetting log backups is the classic "log filled the disk" incident.
  - `BULK_LOGGED` — a special case used during bulk loads.

- **A typical strategy:** weekly full, daily differential, log backups every 5–15 minutes. This is like XtraBackup + binlog streaming, but built in.

```
ALTER DATABASE Sales SET RECOVERY FULL;
BACKUP DATABASE Sales TO DISK = N'/backup/Sales_full.bak' WITH COMPRESSION, CHECKSUM,
INIT;
BACKUP DATABASE Sales TO DISK = N'/backup/Sales_diff.bak' WITH DIFFERENTIAL,
COMPRESSION, CHECKSUM;
BACKUP LOG Sales TO DISK = N'/backup/Sales_log_1415.trn' WITH COMPRESSION, CHECKSUM;
RESTORE VERIFYONLY FROM DISK = N'/backup/Sales_full.bak' WITH CHECKSUM;

-- Point-in-time restore into a new database
RESTORE DATABASE Sales_pitr FROM DISK = N'/backup/Sales_full.bak'
  WITH NORECOVERY, MOVE 'Sales' TO N'/data/Sales_pitr.mdf', MOVE 'Sales_log' TO
N'/data/Sales_pitr.ldf';
RESTORE DATABASE Sales_pitr FROM DISK = N'/backup/Sales_diff.bak' WITH NORECOVERY;
RESTORE LOG Sales_pitr FROM DISK = N'/backup/Sales_log_1415.trn'
  WITH STOPAT = '2026-11-01T14:04:59', RECOVERY;
DBCC CHECKDB (Sales_pitr) WITH NO_INFOMSGS;   -- prove the restored database is
consistent
```

- **Learn next:**
  - **tail-log backups** — back up the last log records before restoring over a damaged database;
  - `COPY_ONLY` backups — ad-hoc backups that don't break the differential chain;
  - backup to URL (Azure Blob, and S3-compatible storage since 2022);
  - backup history in `msdb`.
- **Production standard:** Ola Hallengren's free Maintenance Solution (`DatabaseBackup`, `DatabaseIntegrityCheck`, `IndexOptimize`) run by Agent jobs, plus dbatools `Test-DbaLastBackup` for automated restore tests.

A.7 High availability & disaster recovery

SQL Server feature	What it is	Closest thing you already know
Log shipping	Scheduled log backups restored on a standby	WAL archiving to a warm standby
Always On Availability Group	Database-level replicas, synchronous or asynchronous commit, automatic failover, readable secondaries, a **listener** name	Streaming replication + Patroni + HAProxy (or Group Replication + Router)
Windows Server Failover Cluster quorum (with a file-share or cloud witness)	Decides who is primary	etcd's quorum in Patroni
Failover Cluster Instance	The whole instance fails over between nodes on shared storage	No direct equivalent

SQL Server feature	What it is	Closest thing you already know
Distributed AG	An AG of AGs across sites	InnoDB ClusterSet
Contained AG (2022+)	Logins and Agent jobs travel with the AG	Solves a classic problem: in normal AGs you must sync logins and jobs yourself (dbatools `Copy-DbaLogin`)

On Linux, AGs use **Pacemaker** instead of Windows clustering, and AD integration takes more setup. Most enterprises run SQL Server HA on Windows.

A.8 Automation

- **dbatools** (PowerShell, free, 600+ commands) is the SQL Server DBA's main automation tool: `Backup-DbaDatabase`, `Restore-DbaDatabase`, `Test-DbaLastBackup`, `Copy-DbaLogin`, `Invoke-DbaQuery`, `Get-DbaDbSpace`, and more.
- **Ansible:** the `microsoft.sql` collection installs and configures SQL Server on Linux. `community.general.mssql_db` / `mssql_script` manage databases and run T-SQL. Windows hosts are managed over WinRM with `ansible.windows`, often by calling dbatools through `win_powershell`.
- Reuse your Part 4 ideas: idempotent roles, Vault for secrets, `--check --diff`, rolling patching (patch secondaries → fail over → patch the old primary).

A.9 Indexing, performance and partitioning

- **Method:** start with **wait statistics** (`sys.dm_os_wait_stats`: what is the server waiting on?), then the top queries (**Query Store**, enabled by default for new databases since 2022), then the plans (actual execution plan, `SET STATISTICS IO, TIME ON`).
- **Indexes:**
  - the clustered index *is* the table, like InnoDB (heaps also exist);
  - nonclustered indexes with `INCLUDE` = covering indexes;
  - **filtered** indexes = PostgreSQL partial indexes;
  - **columnstore** indexes for analytics.
- Treat "missing index" suggestions as hints, not orders.
- **Problems specific to SQL Server:**
  - **parameter sniffing** — a cached plan built for one parameter value is bad for others;
  - blocking under default locking (consider RCSI);
  - tempdb contention;
  - MAXDOP / cost threshold settings.
- **Maintenance:** keep statistics fresh (`UPDATE STATISTICS`). Index rebuilds matter less on SSDs than old advice says. Use Ola Hallengren's `IndexOptimize` with sensible thresholds.
- **Partitioning** (instant archiving with `SWITCH`, like MySQL `EXCHANGE PARTITION` / PostgreSQL `DETACH`):

```
CREATE PARTITION FUNCTION pf_month (datetime2)
  AS RANGE RIGHT FOR VALUES ('2026-11-01', '2026-12-01', '2027-01-01');
CREATE PARTITION SCHEME ps_month AS PARTITION pf_month ALL TO ([PRIMARY]);
CREATE TABLE dbo.orders (
```

```
  id bigint IDENTITY, customer_id int, created_at datetime2 NOT NULL, total
decimal(12,2),
  CONSTRAINT pk_orders PRIMARY KEY (id, created_at)
) ON ps_month (created_at);

ALTER PARTITION SCHEME ps_month NEXT USED [PRIMARY];
ALTER PARTITION FUNCTION pf_month() SPLIT RANGE ('2027-02-01');      -- add next
month
ALTER TABLE dbo.orders SWITCH PARTITION 1 TO dbo.orders_archive;     -- instant
archive
```

A.10 SQL Server labs (weeks 48–49)

1. Install SQL Server 2025 Developer in Docker and on the Windows VM. Apply the A.3 settings and explain each one.
2. Recreate the 1.4 role pattern with logins, users and schema grants. Restore the database onto another instance and fix the orphaned users.
3. Rebuild the Part 2 HR mini-system in T-SQL, with a statement-level audit trigger tested on a 1,000-row update.
4. Set the FULL recovery model, then skip log backups and watch the log grow. Then set up Ola Hallengren jobs and do a PITR to one second before a `DROP TABLE`.
5. Build a 2-node Availability Group with a listener (Windows cluster or Pacemaker). Fail over while an app is connected.
6. Run Brent Ozar's `sp_Blitz` and `sp_BlitzCache`. Fix one parameter-sniffing problem and one blocking problem (with RCSI).
7. Automate steps 1–4 with dbatools or Ansible.

A.11 Courses and certification

- **Microsoft Learn** (free): the SQL Server and Azure SQL administration learning paths. They are the official preparation for **DP-300 (Microsoft Certified: Azure Database Administrator Associate)**, which covers SQL Server and Azure SQL, with free yearly online renewal.
- **Brent Ozar** (free): the First Responder Kit (`sp_Blitz` family) and his free "How to Think Like the SQL Server Engine" class, the best short explanation of how SQL Server reads data.
- **Ola Hallengren's Maintenance Solution** and **dbatools.io**: free tools with good documentation and videos.

Sources: SQL Server 2025 GA announcement; SQL Server 2025 on Ubuntu 24.04 (CU1); SQL Server 2025 edition changes; Microsoft SQL certification path 2026.